DARTS: A Lattice-Based Digital Signature Scheme

Algorithm Specification and Supporting Documentation

Yanbin Pan, Qian Lu, Yihui Wu, Jie Li, Chi Cheng, Renzhang Liu, Qian Du

June 29, 2026

Contents

1 Algorithm Specification **3**
- 1.1 Preliminaries and notation . . . 3
- 1.2 Specification . . . 7
- 1.3 Correctness . . . 9
- 1.4 Number of Repetitions in Signature . . . 11
- 1.5 Parameter sets . . . 12

2 Design Rationale **13**

3 Security Analysis **14**
- 3.1 Hardness assumption . . . 15
- 3.2 Security definition . . . 18
- 3.3 Security reduction . . . 19
  - 3.3.1 Security Properties of the Underlying CID . . . 19
  - 3.3.2 UF-CMA Security . . . 23
  - 3.3.3 The Addition of the Strong Unforgeabilty Property . . . 24
- 3.4 Analysis with respect to known attacks . . . 25

4 Performance Evaluation **26**
- 4.1 Reference implementation . . . 26
- 4.2 AVX2 optimized implementation . . . 27
- 4.3 NEON optimized implementation . . . 29

5 Key Properties **29**

1 Algorithm Specification

DARTS is a digital signature scheme based on the Fiat-Shamir with Abort paradigm [16] [17], which incorporates bimodal rejection sampling [11] to improve the efficiency. The security is mainly based on the hardness of the module versions of the lattice problems LWE and SIS [15].

In Subsection 1.1, we introduce the preliminaries and notation. In Subsection 1.2, we present the complete description of our signature scheme. Subsection 1.3 then provides the correctness proof of our signature. Subsequently, Subsection 1.4 gives the method to compute the number of signature repetitions. Finally, Subsection 1.5 summarizes the parameter sets we propose for different security levels.

1.1 Preliminaries and notation

Denote by $\mathbb{R}$, $\mathbb{C}$ and $\mathbb{Z}$ the field of real numbers, the field of complex numbers, and the ring of integers, respectively.

Residue class ring For a positive odd integer q, $\mathbb{Z}_q$ denotes the residue class ring $\mathbb{Z}/q\mathbb{Z}$. For a positive odd integer q and any $t \in \mathbb{Z}$, $t \bmod q$ denotes the unique $r \in \{0, 1, \ldots, q-1\}$ such that $t \equiv r \pmod{q}$, and $t \bmod \pm q$ denotes the unique $r \in \left[-\frac{q-1}{2}, \frac{q-1}{2}\right]$ such that $t \equiv r \pmod{q}$.

Polynomial rings Let n be a power-of-two integer and define the polynomial ring $\mathcal{R} = \mathbb{Z}[x]/(x^n+1)$; for any positive integer q, we further define the quotient ring $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n+1)$.

Coefficient Representation An element $a \in \mathcal{R}_q$ is represented as a polynomial of degree at most $n-1$:

$$a(x) = \sum_{i=0}^{n-1} a_i x^i = a_0 + a_1 x + \cdots + a_{n-1} x^{n-1},$$

where each coefficient $a_i \in \mathbb{Z}_q$. The coefficient vector of a is denoted by $(a_0, a_1, \cdots, a_{n-1}) \in \mathbb{Z}_q^n$.

Incomplete NTT Representation We employ a $(\log_2 n - 2)$-level incomplete NTT to speed up the multiplication in $\mathcal{R}_q$ with degree n.

When prime q satisfies $\frac{n}{2} | (q-1)$ but $n \nmid (q-1)$, the base field $\mathbb{Z}_q$ contains primitive $\frac{n}{2}$-th roots of unity but not primitive n-th roots. Hence, we factor the polynomial X^n+1 into $n/4$ polynomials of degree 4 modulo q. The NTT representation of a polynomial is then a vector of $n/4$ polynomials of degree 4. We define the NTT such that it produces outputs in bit-reversed order, a direct consequence of employing a simple, in-place Cooley-Tukey butterfly structure without an additional permutation stage. Specifically, let ζ be the primitive $\frac{n}{2}$-th root of unity modulo q and $\left\{\zeta, \zeta^3, \zeta^5, \cdots, \zeta^{n/2-1}\right\}$ the set of all the $\frac{n}{2}$-th roots of unity. The polynomial X^n+1 can be written as

$$X^n + 1 = \prod_{i=0}^{n/4-1} (X^4 - \zeta^{2i+1}) = \prod_{i=0}^{n/4-1} (X^4 - \zeta^{2\mathsf{br}_\epsilon(i)+1}),$$

where $\epsilon = \log_2(n/4)$ and $\mathsf{br}_\epsilon(i)$ for $i = 0, \cdots, n/4-1$ is the bit reversal of the unsigned ϵ-bit integer i. Then the NTT of $g \in \mathcal{R}_q$ is given by

$$\left(g \bmod X^4 - \zeta^{2\mathsf{br}_\epsilon(0)+1}, \cdots, g \bmod X^4 - \zeta^{2\mathsf{br}_\epsilon(n/4-1)+1}\right).$$

Multiplication with Karatsuba algorithm Incomplete NTT with Karatsuba algorithm: Subsequently, to further optimize computational efficiency of multiplication in each component ring $\mathcal{R}_q[x]/(X^4 - \zeta^{2i+1})$, we employ the Karatsuba algorithm as in [22]. The primary role of the Karatsuba algorithm here to reduce the number of required modular multiplications from 16 to 9. Hence, it reduces the total modular multiplication count by $7/4n$ through the use of Karatsuba decomposition at $(\log_2 n - 2)$-level.

Matrices and Vectors Matrices are denoted by bold uppercase letters and vectors by bold lowercase letters. The transpose of a vector or matrix is indicated by a superscript $\top$, e.g., $\mathbf{v}^\top$. We use $||\mathbf{v}||_2 = \left(\sum_{i=0}^{n-1} v_i^2\right)^{1/2}$ and $||\mathbf{v}||_\infty = \max_{0 \le i \le n-1} |v_i|$ to denote its Euclidean norm and infinity norm, respectively.

For $\mathbf{v} \in \mathcal{R}_q$, let $\mathbf{v} = (v_0, \ldots, v_{n-1}) \in \mathbb{Z}_q^n$ denote its coefficient vector representation. $\mathbf{A} \in \mathcal{R}_q^{k \times l}$ and $\mathbf{a} \in \mathcal{R}_q^k$ consist of entries sampled from the polynomial ring $\mathcal{R}_q$. The length of $\mathbf{a}$ means the length of the vector that expands $\mathbf{a}$'s every component into its coefficient vector representation.

Hash Functions and Extendable-Output Functions

$\mathsf{H}_{\mathbf{gen}}$**:** Hash function for generating seeds.

H_1**:** Hash function for hashing message.

H_2**:** Hash function with fixed weight output. This function employs an extendable-output function to generate a uniformly random sparse polynomial in $\mathcal{R}_q$, where exactly τ coefficients are set to 1 and the remaining $n - \tau$ coefficients are 0.

Distributions and Sampling

Distribution for Public Key: expandA The function expandA described as Algorithm 1 takes 32-byte (or 64-byte) $\text{seed}_{\mathbf{A}}$ as input and outputs a uniformly random matrix $\mathbf{A} \in \mathcal{R}_q^{k \times (l-1)}$ in NTT domain representation by rejection sampling.

Algorithm 1 ExpandA: Matrix Generation from the seed_A

Require: $\mathsf{seed}_\mathsf{A} \in \{0,1\}^\rho$, dimensions k and $l-1$
Ensure: Matrix $\mathbf{A} \in \mathcal{R}_q^{k \times (l-1)}$ with uniformly random entries

```
for i = 0 to k − 1 do
    for j = 0 to l − 2 do
        nonce ← (i ≪ 8) + j                          ▷ Encode (i, j) as a 16-bit integer
        A[i][j] ← PolyUniform(seed_A, nonce)          ▷ By rejection sampling
    end for
end for
return A
```

Distribution for Secret: expandS This function, described as Algorithm 3, generates a vector $\mathbf{s} \in \mathcal{R}^{k+l}$, whose coefficients s_i are drawn independently from a ternary distribution $\mathcal{T}_p$ in which $\Pr(s_i = 1) = \Pr(s_i = -1) = p$ and $\Pr(s_i = 0) = 1 - 2p$ according to Algorithm 2.

Algorithm 2 PolyTernaryP: Sampling a Ternary Polynomial from Seed and Nonce

Require: $\mathsf{seed} \in \{0,1\}^\rho$, $nonce \in \{0,1\}^{16}$ and distribution parameter $p \in (0, 1/2)$.
Ensure: Polynomial $\mathbf{a} = (a_0, \ldots, a_{N-1})$ with coefficients sampled from $\mathcal{T}_p$, where $a_i \in \{-1, 0, 1\}$.

```
Determine blocks_numbers based on the density parameter p
ctr ← 0
state ← XOF.init(seed ‖ nonce)
buf ← XOF.squeezeblocks(blocks_numbers ‖ state)
ctr ← rej_P(a_0, n, buf)                    ▷ Rejection sampling for target probability p
while ctr < N do
    buf ← XOF.squeezeblocks(1 ‖ state)
    ctr ← rej_P(a_ctr, n − ctr, buf)
end while
return a
```

Algorithm 3 ExpandS: Sample $\mathbf{s} = (s_0, \mathbf{s_1}, \mathbf{e}_1)$ from the $\mathsf{seed_s}$

Require: $\mathsf{seed_s} \in \{0,1\}^\rho$
Ensure: $(s_0, \mathbf{s_1}, \mathbf{e}_1) \in \mathcal{R}_q^{1+(l-1)+k}$ with ternary coefficients distributed as $\mathcal{T}_p$
1: $\text{counter}_\mathbf{s} = 0$
2: $s_0 \leftarrow \text{PolyTernaryP}(\mathsf{seed_s}, \text{counter}_\mathbf{s})$
3: $\text{counter}_\mathbf{s} \leftarrow \text{counter}_\mathbf{s} + 1$
4: **for** $i = 0$ **to** $l - 2$ **do**
5: $\quad \mathbf{s_1}[i] \leftarrow \text{PolyTernaryP}(\mathsf{seed_s}, \text{counter}_\mathbf{s})$
6: $\quad \text{counter}_\mathbf{s} \leftarrow \text{counter}_\mathbf{s} + 1$
7: **end for**
8: **for** $i = 0$ **to** $k - 1$ **do**
9: $\quad \mathbf{e}[i] \leftarrow \text{PolyTernaryP}(\mathsf{seed_s}, \text{counter}_\mathbf{s})$
10: $\quad \text{counter}_\mathbf{s} \leftarrow \text{counter}_\mathbf{s} + 1$
11: **end for**
12: **return** $(s_0, \mathbf{s_1}, \mathbf{e}_1)$

Rejection on Secret Our scheme incorporates a rejection sampling step on secret $\mathbf{s}$ to decrease the signing rejection probability.

Denote by τ the parameter for H_2, that is, the number of non-zero coefficients of its output $\mathbf{c}$, $m = \lfloor n/\tau \rfloor$ and $r = n \bmod \tau$. Let ω_j (for $1 \le j \le n$) be the primitive $2n$-th roots of unity in $\mathbb{C}$. For a secret key $\mathbf{s} = (\mathbf{s}_1, \ldots, \mathbf{s}_{k+l}) \in \mathcal{R}^{k+l}$, its evaluation at ω_j is defined as $\mathbf{s}(\omega_j) = (\mathbf{s}_1(\omega_j), \ldots, \mathbf{s}_{k+l}(\omega_j)) \in \mathbb{C}^{k+l}$. We first compute the value $\mathcal{N}(\mathbf{s})$ about secret as follows:

$$\mathcal{N}(\mathbf{s}) := \tau \sum_{i=1}^{m} \max_{j}^{i-th} \|\mathbf{s}(\omega_j)\|_2^2 + r \cdot \max_{j}^{(m+1)-th} \|\mathbf{s}(\omega_j)\|_2^2.$$

Sequently, we verify whether the condition $\mathcal{N}(\mathbf{s}) \le \gamma^2 n$ holds, where γ is an empirical heuristic parameter that can be determined by the methodology from [8]. By Lemma 3 in [8], this condition guarantees that for any c that H_2 outputs, $\|c\mathbf{s}\|_2 \le \gamma\sqrt{\tau}$.

To determine γ, we perform large-scale sampling of the secret vector $\mathbf{s}$ and calculate the corresponding values of $\mathcal{N}(\mathbf{s})$. After sorting these values in ascending order, we select a threshold $N_{\max}(\mathbf{s})$ that corresponds to a predefined acceptance ratio (e.g., 50%), and subsequently set $\gamma = \sqrt{N_{\max}(\mathbf{s})/n}$.

Distribution for Samples: expandY For any $r > 0$ and $\mathbf{c} \in \mathbb{R}^m$, the m-dimensional Euclidean ball $\mathcal{B}_{\mathbb{R},m}(r, \mathbf{c})$ is defined as $\{\mathbf{x} \in \mathbb{R}^m \mid \|\mathbf{x} - \mathbf{c}\|_2 \le r\}$, where the center $\mathbf{c}$ is omitted if $\mathbf{c} = \mathbf{0}$. Furthermore, for a positive integer N, the discrete hyperball is defined by $\mathcal{B}_{(1/N)\mathbb{R},m}(r, \mathbf{c}) = (1/N)\mathbb{R}^m \cap \mathcal{B}_{\mathbb{R},m}(r, \mathbf{c})$. To generate samples, we adopt the method from [19] and [8]. Specifically, we define the function $\mathsf{expandY}$ to sample a vector $\mathbf{y}$ uniformly from the hyperball $\mathcal{B}_{(1/N)\mathbb{R},k+l}(B)$.

Distribution for Signature: Double Sphere Distribution Building on the findings in [9], it is observed that bimodal rejection sampling [11] over a hyperball achieves a lower rejection rate than its unimodal counterpart. Drawing on this methodology, we define the following double sphere distribution.

For radii $B_0 < B'$, let $\mathcal{B}_{(1/N)\mathbb{R},k+l}(B')$ and $\mathcal{B}_{(1/N)\mathbb{R},k+l}(B_0)$ be two concentric hyperballs centered at the origin. We define the Double Sphere Distribution, denoted by $\mathcal{B}_\mathcal{B}(B_0, B')$, as a mixture of two uniform distributions over $\mathcal{B}_{(1/N)\mathbb{R},k+l}(B_0)$ and the region $\mathcal{B}_{(1/N)\mathbb{R},k+l}(B') \setminus \mathcal{B}_{(1/N)\mathbb{R},k+l}(B_0)$, with total probabilities p_1 and p_2 respectively. Moreover, it satisfies condition $p_1 = 2p_2$. The corresponding density function is as follows: The corresponding density function is as follows:

$$f(\mathbf{z}) = \begin{cases} p_1 & \mathbf{z} \in \mathcal{B}_{(1/N)\mathbb{R},k+l}(B_0), \\ \frac{1}{2}p_1 & \mathbf{z} \in \mathcal{B}_{(1/N)\mathbb{R},k+l}(B') \setminus \mathcal{B}_{(1/N)\mathbb{R},k+l}(B_0). \end{cases}$$

See Figure 1.

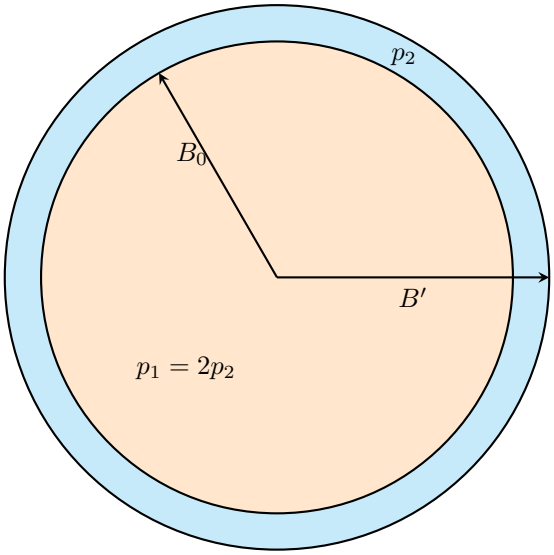


Figure 1: Double Sphere Distribution

Compression and Decompression To reduce the size of signatures, we apply a modulus switching technique to certain components. Let q be the original modulus and let d be a positive integer. For $r \in \mathbb{Z}_q$ and $r' \in \{0, \cdots, 2^d - 1\}$, compression and decompression are defined as

$$\mathsf{Comp}_d(r) = \left\lfloor \frac{2^d}{q} \cdot r \right\rceil \bmod 2^d, \qquad \mathsf{Decomp}_d(r') = \left\lfloor \frac{q}{2^d} \cdot r' \right\rceil \bmod q,$$

where $\lfloor \cdot \rceil$ denotes rounding to the nearest integer. For any $x \in \mathbb{Z}_q$, let

$$x' = \mathsf{Decomp}_d(\mathsf{Comp}_d(x)).$$

The reconstruction error $e = (x - x') \bmod \pm q$ is bounded by

$$|e| \leq \left\lfloor \frac{q}{2^{d+1}} \right\rceil.$$

For vectors or polynomials, the above operations are applied independently to each coefficient.

Encoding and Decoding Range Asymmetric Numeral Systems (rANS) [12] is an entropy coding technique that combines the high compression efficiency of arithmetic coding with the exceptional throughput speeds of Huffman coding. It is particularly well-suited for scenarios where symbol probability distributions are highly non-uniform. Subsequently, Espitau et al. [13] and Cheon [8] adopted rANS to further compress the size of signature. Hence, we directly employ the rANS encoder described in Algorithm 4 and 5.

Definition 1 (Range Asymmetric Numeral System, rANS)**.** Let $S \subseteq [0, 2^n - 1]$, where $n \in \mathbb{N}$, and let a function $g : S \to \mathbb{Z} \cap (0, 2^n]$ satisfy

$$\sum_{s \in S} g(s) \leq 2^n$$

and for all $s \notin S$, $g(s) = 0$. Then (S, g) is called a Range Asymmetric Numeral System.

Given the Range Asymmetric Numeral System (S, g), define the following functions:

1. **Cumulative Distribution Function (CDF)**: $\mathrm{CDF} : S \to \mathbb{Z}$, defined as $\mathrm{CDF}(s) = \sum_{i=0}^{s-1} g(i)$, where $\mathrm{CDF}(0) = 0$.
2. **Symbol Function**: $\mathrm{symbol} : \mathbb{Z} \to S$, defined as $\mathrm{symbol}(x) = s \iff \mathrm{CDF}(s) \leq x < \mathrm{CDF}(s+1)$. This function is primarily used during the decoding process to determine the interval of the encoded value x for subsequent decoding steps.

The encoding function C is defined as:

$$C(x, s) = \left\lfloor \frac{x}{g(s)} \right\rfloor \cdot 2^n + (x \bmod g(s)) + \mathrm{CDF}(s)$$

For the set S and the normalized frequencies $g/2^n$, the rANS encoding and decoding functions are defined as above.

Algorithm 4 rANS_Encode: Encode a sequence $\mathbf{z}$ into state x

Require: Sequence of symbols $\mathbf{z} = (z_1, \ldots, z_m)$
Ensure: Compressed state x
1: $x \leftarrow 0$
2: **for** $i = m$ **down to** 1 **do**
3: $\quad s \leftarrow z[i]$
4: $\quad x \leftarrow C(x, s)$
5: **end for**
6: **return** x

Algorithm 5 rANS_Decode: Recover symbols from state x

Require: Compressed state x, length m
Ensure: Recovered symbols $(t_1, \ldots, t_m)$
1: $y \leftarrow x$
2: $M \leftarrow 2^n$
3: **for** $i = 1$ **to** m **do**
4: $\quad slot \leftarrow y \bmod M$
5: $\quad t_i \leftarrow \mathrm{symbol}(slot)$
6: $\quad y \leftarrow g(t_i) \cdot \lfloor y/M \rfloor + slot - \mathrm{CDF}(t_i)$
7: **end for**
8: **return** $(t_1, \ldots, t_m)$

1.2 Specification

In Algorithms 6, 7 and 8 we define key generation, signing, and verification of DARTS. For clarity, we provide the definitions of the parameters for these algorithms in Table1.

Table 1: System Parameters and Notations

Parameter	Description
λ	Security parameter.
$\rho(\cdot)$	Length function for random seeds.
n	Degree of the polynomial ring $\mathcal{R}, \mathcal{R}_q$.
q	Modulus of the coefficient ring.
k, l	Dimensions of the signature components $\mathbf{z}_2$ and $\mathbf{z}_1$, respectively.
τ	Number of non-zero coefficients in the challenge polynomial.
p	Probability of $1(-1)$ for each coefficient of $\mathbf{s}$ (ternary: $\mathcal{T}_p(0) = 1 - 2p$).
γ	Heuristic parameter used in rejection sampling for the secret vector.
$B_{\mathbf{s}c}$	Upper bound on the norm of $\mathbf{s}c$.
d	Size of the compression factor.
B, B', B_0	Radii for $\mathbf{y}$-sampling (B) and rejection sampling (B', B_0).
B_{SIS}	Bound employed in the verification algorithm (SIS-based).
α	The vector $\alpha = (1, 0, \cdots, 0) \in \mathcal{R}^k$.
$M_{\mathsf{reject}}, M_{\mathsf{comp}}$	Expected restarts from rejection sampling, and hint generation, respectively.
M	Expected number of calls $\mathsf{expandY}$ to function per signature.

Key Generation To ensure the correctness of our signature scheme based on bimodal rejection sampling over hyperballs, the key pair $(\mathbf{A}, \mathbf{s}) \in \mathcal{R}_q^{k \times (k+\ell)} \times \mathcal{R}_q^{k+\ell}$ should satisfy

$$\mathbf{As} \equiv \frac{q+1}{2}\alpha \pmod{q}. \tag{1}$$

The generation of the key pair $(\mathbf{A}, \mathbf{s})$ satisfying Eq. (1) is carried out as follows. First, the key generation algorithm samples an MLWE instance $\mathbf{b} \equiv \mathbf{A}_1\mathbf{s}_1 + \mathbf{e}_1 \mod q$, where $\mathbf{A}_1 \xleftarrow{\$} \mathcal{R}_q^{k\times(l-1)}$ and $(\mathbf{s}_1, \mathbf{e}_1) \in S_\eta^{l-1} \times S_\eta^k$ are small vectors. Second, it generates an invertible vector $\mathbf{s}_0 \in \mathcal{R}_q$ with small norm and compute $\mathbf{A}_0 = -\mathbf{s}_0^{-1}(\mathbf{b} - \frac{q+1}{2}\alpha)$. Then it makes the key pair $(\mathbf{A}, \mathbf{s})$ as

$$\mathbf{A} \equiv (\mathbf{A}_0, \mathbf{A}_1, \mathbf{I}) \mod q,$$

$$\mathbf{s} = (\mathbf{s}_0, \mathbf{s}_1, \mathbf{e}_1).$$

Next, we compute $\mathcal{N}(\mathbf{s})$ for the secret $\mathbf{s}$ and check whether $\mathcal{N}(\mathbf{s}) \le \gamma^2 n$ holds. Only when this holds can we guarantee $\|\mathbf{s}c\|_2 \le \gamma\sqrt{\tau}$. Otherwise, the secret key $\mathbf{s}$ must be resampled until the inequality is satisfied.

Algorithm 6 KeyGen

Require: Security parameter 1^λ
Ensure: Secret key sk, public key pk
1: $\text{seed} \leftarrow \{0,1\}^{\rho(1^\lambda)}$
2: $(\text{seed}_\mathbf{A}, \text{seed}_\mathbf{s}, K) \leftarrow \mathsf{H}_{\text{gen}}(\text{seed})$
3: $\mathbf{A}_1 \leftarrow \text{expandA}(\text{seed}_\mathbf{A}) \in \mathcal{R}_q^{k\times(l-1)}$
4: $\text{counter}_\mathbf{s} = 0$
5: $(s_0, \mathbf{s}_1, \mathbf{e}_1) \leftarrow \text{expandS}(\text{seed}_\mathbf{s}, \text{counter}_\mathbf{s}) \in \mathcal{R}_q \times \mathcal{R}_q^{l-1} \times \mathcal{R}_q^k$, $\text{counter}_\mathbf{s} = \text{counter}_\mathbf{s} + 1$
6: If s_0 is not invertible or $\mathcal{N}(\mathbf{s}) > \gamma^2 n$ then **Go to** step 5
7: $\mathbf{A}_0 \leftarrow -s_0^{-1}\left(\mathbf{A}_1 \cdot \mathbf{s}_1 + \mathbf{e}_1 - \frac{q+1}{2}\alpha^\top\right) \in \mathcal{R}_q^k$
8: $\mathbf{A} \leftarrow (\mathbf{A}_0, \mathbf{A}_1, \mathbf{I}_k) \in \mathcal{R}_q^{k\times(k+l)}$
9: $\mathbf{s} \leftarrow (s_0, \mathbf{s}_1, \mathbf{e}_1)$
10: **return** $(\text{pk} = (\mathbf{A}_0, \text{seed}_\mathbf{A}), \text{sk} = (\text{pk}, \mathbf{s}, K))$

Signature The signing process proceeds as follows. First, the public key matrix $\mathbf{A}$ is reconstructed from $\text{seed}_\mathbf{A}$ and $\mathbf{A_0}$. Then, the masking vector $\mathbf{y} = (\mathbf{y_1}, \mathbf{y_2}) \in \mathcal{R}_q^{k+l}$ is sampled uniformly from the hyperball $\mathcal{B}_{(1/N)\mathbb{R},k+l}(B)$. The commitment is computed by multiplying the public matrix $\mathbf{A}$ by the masking vector $\mathbf{y}$ after it has been rounded to the nearest integer, denoted as $\omega \equiv \mathbf{A}\lfloor\mathbf{y}\rceil \pmod{q}$. Next, the challenge $c \in \mathcal{R}_q$ is derived from the high-order bits of ω and the message μ. It contains exactly τ non-zero entries (all ones) to ensure sufficient entropy against exhaustive search. The potential signature is then generated as $\mathbf{z} = \mathbf{y} + (-1)^b\mathbf{s}c$, where $b \in \{0,1\}$. Subsequently, rejection sampling is applied to $\mathbf{z}$ to make its distribution independent of the secret information for preventing leakage. Hence, the resulting distribution follows the Double Sphere Distribution.

After passing rejection sampling, the signature is further compressed for compactness. Specifically, instead of transmitting $\mathbf{z}_2$ directly, a compact hint $\mathbf{h}$ is generated by using $\mathbf{z}_2$ and ω. To prevent potential information leakage, the condition $\mathsf{Comp}_d(\omega) = \mathsf{Comp}_d(\omega + (1-2b)c\alpha)$ is strictly enforced for $b \in \{0,1\}$. Meanwhile, modulo switching is applied to $\mathbf{z}_1$ to reduce its size. Experimental results indicate that the coefficient distributions of both the hint $\mathbf{h}$ and $\mathsf{Comp}_d(\lfloor\mathbf{z}_1\rceil)$ are highly non-uniform and closely concentrated. With this property, we employ rANS to encode $\mathbf{h}$ and $\mathsf{Comp}_d(\lfloor\mathbf{z}_1\rceil)$. These steps are essential for minimizing communication overhead.

Algorithm 7 Sign

Input: $\mathrm{sk} = (\mathrm{pk}, \mathbf{s}, K)$, M
Output: Signature $\sigma = (c, \mathbf{e}_z, \mathbf{x}, \mathsf{Encode}(\mathsf{h}))$

Reconstruct $\mathbf{A} = (\mathbf{A}_0, \mathbf{A}_1, \mathbf{I}_k)$ from pk
$\mu \leftarrow \mathsf{H}_1(\mathrm{pk}, M)$
$\mathrm{seedbuf} = \mathsf{H}_1(K, \mu)$
$\mathrm{counter}_{\mathbf{y}} = 0$
$(\mathbf{y}, b, b') \leftarrow \mathrm{expendY}(\mathrm{seedbuf}, \mathrm{counter}_{\mathbf{y}})$ ▷ where $\mathbf{y} = (\mathbf{y}_1, \mathbf{y}_2) \in \mathcal{R}_q^{l+k}$
$\mathrm{counter}_{\mathbf{y}} = \mathrm{counter}_{\mathbf{y}} + 1$
$\omega \equiv \mathbf{A} \lfloor \mathbf{y} \rceil \pmod q$
$c = \mathsf{H}_2(\mathsf{Comp}_d(\omega), \mu)$
if $\mathsf{Comp}_d(\omega) \neq \mathsf{Comp}_d(\omega + (1-2b)c\alpha)$
  then **Go to** step 5
$\mathbf{z} = (\mathbf{z}_1, \mathbf{z}_2) = \mathbf{y} + (-1)^b \mathbf{s}c$
if $\|\mathbf{z}\|_2 > B'$ or $(\|2\mathbf{z} - \mathbf{y}\|_2 < B, \|\mathbf{z}\|_2 > B_0, b' = 0)$ then
  then **Go to** step 5
$\mathbf{h} \equiv \mathsf{Comp}_d(\omega) - \mathsf{Comp}_d(\omega - \lfloor \mathbf{z}_2 \rceil + (1-b)c\alpha) \pmod{2^d}$
$\mathbf{x} = \mathsf{Encode}(\mathsf{Comp}_d(\lfloor \mathbf{z}_1 \rceil))$ ▷ rANS_Encode
$\mathbf{e}_z = \lfloor \mathbf{z}_1 \rceil - \mathsf{Decomp}_d(\mathsf{Comp}_d(\lfloor \mathbf{z}_1 \rceil))$
return $\sigma = (c, \mathbf{e}_z, \mathbf{x}, \mathsf{Encode}(\mathsf{h}))$

Verification The verification procedure consists of two primary stages. In the first stage, the high bits $\tilde{\omega}_1$ are recovered, followed by the recomputation of the challenge c' using $\tilde{\omega}_1$. The algorithm then verifies whether the recomputed challenge c' matches the received c. If they do not match, verification fails; otherwise, the process proceeds to the second stage. In the second stage, the signature $\tilde{\mathbf{z}}$ is reconstructed and checked against the norm bound $||\tilde{\mathbf{z}}||_2 < B''$. The signature is accepted only if this condition is also satisfied.

Algorithm 8 Verify

Input: pk, M, $\sigma = (c, \mathbf{e}_z, \mathbf{x}, \mathsf{Encode}(\mathsf{h}))$
Output: 1 (Accept) or 0 (reject)

$\tilde{\mathbf{z}}_1 \leftarrow \mathsf{Decomp}_d(\mathbf{x}) + \mathbf{e}_z$
$\tilde{\mathbf{h}} \leftarrow \mathsf{Decode}(\mathsf{Encode}(\mathbf{h}))$
$\mathbf{A}_1 \leftarrow \mathsf{expandA}(\mathrm{seed}_{\mathbf{A}})$
$\bar{\mathbf{A}} \leftarrow (\mathbf{A}_0, \mathbf{A}_1) \bmod q$
$\tilde{w}_1 \leftarrow \tilde{\mathbf{h}} + \mathsf{Comp}_d\big(\bar{\mathbf{A}}\tilde{\mathbf{z}}_1 + \frac{q+1}{2}c\alpha\big) \pmod q$
$\tilde{\mu} \leftarrow \mathsf{H}_1(\mathrm{pk}, M)$
if $c \neq \mathsf{H}_2(\tilde{w}_1, \tilde{\mu})$ **then**
  return 0
end if
$\tilde{\mathbf{z}}_2 \leftarrow \mathsf{Decomp}_d(\tilde{w}_1) - \big(\bar{\mathbf{A}}\tilde{\mathbf{z}}_1 + \frac{q+1}{2}c\alpha\big) \pmod q$
$\tilde{\mathbf{z}} \leftarrow (\tilde{\mathbf{z}}_1, \tilde{\mathbf{z}}_2)$
return $\big(\|\tilde{\mathbf{z}}\| < B''\big)$

1.3 Correctness

Theorem 1. Let $B'' = B' + \frac{\sqrt{n(k+l)}}{2} + \sqrt{nk}\left\lfloor \frac{q}{2^{d+1}} \right\rceil$. Under this choice of B'', the scheme satisfies correctness. That is, for any message M and any key pair $(\mathrm{pk}, \mathrm{sk})$ generated by $\mathsf{KeyGen}(1^\lambda)$,

$$\mathsf{Verify}(\mathrm{pk}, M, \mathsf{Sign}(\mathrm{sk}, M)) = 1.$$

Proof. Note that Step 7 and Step 12 in Algorithm 8 are the only two steps that decides to accept or reject the signature. We next proceed to carry out the verification in two parts.

Check Step 7. From the construction of key pair we can get :

$$\begin{aligned}\bar{\mathbf{A}}\tilde{\mathbf{z}}_1 + \frac{q+1}{2}\alpha c &\equiv \bar{\mathbf{A}}\lfloor \mathbf{z}_1 \rceil + \lfloor \mathbf{z}_2 \rceil + \frac{q+1}{2}\alpha c - \lfloor \mathbf{z}_2 \rceil \\ &\equiv [\bar{\mathbf{A}}, \mathbf{I}]\lfloor \mathbf{z} \rceil + \frac{q+1}{2}\alpha c - \lfloor \mathbf{z}_2 \rceil \\ &\equiv \mathbf{A}\lfloor \mathbf{y} \rceil + (-1)^b \mathbf{A}\mathbf{s}c + \frac{q+1}{2}\alpha c - \lfloor \mathbf{z}_2 \rceil \\ &\equiv \omega + [1 + (-1)^b]\frac{q+1}{2}\alpha c - \lfloor \mathbf{z}_2 \rceil \\ &\equiv \omega + (1-b)\alpha c - \lfloor \mathbf{z}_2 \rceil \pmod{q}. \end{aligned} \tag{2}$$

Therefore, we can recover the challenge c using $\mathbf{h}$ as follows:

$$\begin{aligned}\tilde{w}_1 &= \mathbf{h} + \mathsf{Comp}_d(\bar{\mathbf{A}}\tilde{\mathbf{z}}_1 + \frac{q+1}{2}\alpha c) \\ &\equiv \mathsf{Comp}_d(\omega) - \mathsf{Comp}_d(\omega - \lfloor \mathbf{z}_2 \rceil + (1-b)c\alpha) + \mathsf{Comp}_d(\bar{\mathbf{A}}\tilde{\mathbf{z}}_1 + \frac{q+1}{2}\alpha c) \pmod{2^d} \\ &= \mathsf{Comp}_d(w).\end{aligned}$$

Hence, we have

$$\mathsf{H}_2(\tilde{w}_1, \tilde{\mu}) = \mathsf{H}_2(\mathsf{Comp}_d(w), \mu) = c.$$

Check Step 12. During the reconstruction of the challenge c, $\lfloor \mathbf{z}_2 \rceil$ can be derived from Eq. (2) as

$$\lfloor \mathbf{z}_2 \rceil \equiv \omega + (1-b)\alpha c - (\bar{\mathbf{A}}\tilde{\mathbf{z}}_1 + \frac{q+1}{2}\alpha c) \pmod{q}.$$

Simultaneously, during the verification procedure, $\tilde{\mathbf{z}}_2$ can be derived as

$$\begin{aligned}\tilde{\mathbf{z}}_2 &\equiv \mathsf{Decomp}_d(\tilde{w}_1) - \left(\bar{\mathbf{A}}\tilde{\mathbf{z}}_1 + \frac{q+1}{2}c\alpha\right) \\ &\equiv \mathsf{Decomp}_d(\mathsf{Comp}_d(\omega)) - \left(\bar{\mathbf{A}}\tilde{\mathbf{z}}_1 + \frac{q+1}{2}c\alpha\right) \\ &\equiv \omega - \tilde{e} - \left(\bar{\mathbf{A}}\tilde{\mathbf{z}}_1 + \frac{q+1}{2}c\alpha\right) \pmod{q},\end{aligned}$$

where the error

$$\tilde{e} = \omega - \mathsf{Decomp}_d(\mathsf{Comp}_d(\omega)).$$

Therefore, we can get that

$$\lfloor \mathbf{z}_2 \rceil - \tilde{\mathbf{z}}_2 \equiv (1-b)\alpha c + \tilde{e} \pmod{q}. \tag{3}$$

Since the signing procedure ensures that the commitment ω satisfies that:

$$\mathsf{Comp}_d(\omega) = \mathsf{Comp}_d(\omega + (1-2b)c\alpha). \tag{4}$$

Considering the error $e_{\alpha c}$ introduced by compressing $\omega + (1-b)c\alpha$, together with Eq. (4), we have

$$\begin{aligned}e_{\alpha c} &= \omega + (1-b)c\alpha - \mathsf{Decomp}_d(\mathsf{Comp}_d(\omega + (1-b)c\alpha)) \\ &= \omega + (1-b)c\alpha - \mathsf{Decomp}_d(\mathsf{Comp}_d(\omega)) \\ &= (1-b)\alpha c + \tilde{e}.\end{aligned}$$

Due to the compression, it holds that

$$\|e_{\alpha c}\|_\infty \leq \left\lfloor \frac{q}{2^{d+1}} \right\rceil,$$

then we have

$$\|\lfloor \mathbf{z}_2 \rceil - \tilde{\mathbf{z}}_2\|_\infty = \|(1-b)\alpha c + \tilde{e}\|_\infty = \|e_{\alpha c}\|_\infty \leq \left\lfloor \frac{q}{2^{d+1}} \right\rceil.$$

Subsequently, we utilize results above to evaluate the ℓ_2-norm of $\tilde{\mathbf{z}}$ reconstructed during verification procedure.

$$\begin{aligned}
\|\tilde{\mathbf{z}}\|_2 &= \|-\mathbf{z} + \mathbf{z} - \lfloor \mathbf{z} \rceil + \lfloor \mathbf{z} \rceil + \tilde{\mathbf{z}}\|_2 \\
&\leq \|\mathbf{z}\|_2 + \|\mathbf{z} - \lfloor \mathbf{z} \rceil\|_2 + \|\lfloor \mathbf{z} \rceil - \tilde{\mathbf{z}}\|_2 \\
&\leq B' + \sqrt{n(k+l)}\,\|\mathbf{z} - \lfloor \mathbf{z} \rceil\|_\infty + \|\lfloor \mathbf{z}_2 \rceil - \tilde{\mathbf{z}}_2\|_2 \\
&\leq B' + \frac{\sqrt{n(k+l)}}{2} + \sqrt{nk}\,\|\lfloor \mathbf{z}_2 \rceil - \tilde{\mathbf{z}}_2\|_\infty \\
&\leq B' + \frac{\sqrt{n(k+l)}}{2} + \sqrt{nk}\left\lfloor \frac{q}{2^{d+1}} \right\rceil.
\end{aligned}$$

The definition of B'' implies that the scheme is correct. □

1.4 Number of Repetitions in Signature

This section analyzes the expected number of repetitions required during the signing process. Specifically, this repetition count refers to the frequency of sampling the masking vector $\mathbf{y}$ from a uniform distribution over the hyperball. Repetitions occur for two primary reasons:

- The condition of bimodal rejection sampling is not satisfied, or
- The resulting signature fails to meet the requirements of the hint $\mathbf{h}$.

In the following, we provide a detailed derivation of the repetition rate contributed by each of these factors.

Expected Number M_{reject} of Repetitions in Bimodal Rejection Sampling First, we present a formal definition for the bimodal rejection sampling used in DARTS.

Definition 2 (Bimodal Rejection Sampling over Hyperballs)**.** Let n be the degree of the ring $\mathcal{R}$, and $v, k, l, B, B', B_0 > 0$ be parameters satisfying $B'^2 + v^2 \leq B^2$ and $B_0 = \min\{B', B - v\}$. Suppose $\mathbf{t} \leftarrow \mathcal{B}_{(1/N)\mathbb{R},k+l}(0, v)$, where the precision parameter N is chosen to satisfy the requirements for uniform sampling over discrete hyperballs and bimodal rejection sampling as established in [19, 8]. Let $\mathbf{y} \xleftarrow{\$} \mathcal{B}_{(1/N)\mathbb{R},k+l}(0, B)$, and $\mathbf{z} = \mathbf{y} + (-1)^b\mathbf{t}$ for a uniform bit $b \in \{0, 1\}$. We define the acceptance probability function $p(\mathbf{z}) : \mathcal{R}^{k+l} \to \{0, 1/2, 1\}$ such that:

$$p(\mathbf{z}) = \begin{cases} 0 & \text{if } \|\mathbf{z}\|_2 \geq B', \\ 1/2 & \text{else if } (\|\mathbf{z} - \mathbf{t}\|_2 < B) \wedge (\|\mathbf{z} + \mathbf{t}\|_2 < B) \wedge (\|\mathbf{z}\|_2 > B_0), \\ 1 & \text{otherwise.} \end{cases}$$

With $\mathbf{z}$ drawn from the original distribution and $p(\mathbf{z})$ as the acceptance probability, the output follows the double sphere distribution.

For simplicity, we assume that $\mathbf{y}$ is uniformly randomly selected in a ball instead of a discrete ball, then the original distribution of $\mathbf{z}$ follows the density function:

$$g(\mathbf{z}) = \begin{cases} \frac{1}{V(B)} & \text{if } (\|\mathbf{z} - \mathbf{s}c\|_2 < B) \wedge (\|\mathbf{z} + \mathbf{s}c\|_2 < B), \\ \frac{1}{2 \cdot V(B)} & \text{otherwise,} \end{cases}$$

where $V(B) = \frac{\pi^{m/2}}{\Gamma(\frac{m}{2}+1)}B^m$ denotes the volume of an m-dimensional Euclidean ball with radius B for $m = (k + l) \cdot n$.

Note that our goal is to obtain the target distribution (once again consider the continuous instead of the discrete ball) as follows:

$$f(\mathbf{z}) = \begin{cases} p_1 & \mathbf{z} \in \mathcal{B}_{\mathbb{R},m}(0, B_0), \\ \frac{1}{2}p_1 & \mathbf{z} \in \mathcal{B}_{\mathbb{R},m}(0, B') \setminus \mathcal{B}_{\mathbb{R},m}(0, B_0). \end{cases}$$

According to definition of $f(\mathbf{z})$ and the formula for the volume of a hyperball, we get:

$$p_1 \cdot V(B_0) + \frac{1}{2} p_1 \cdot (V(B') - V(B_0)) = 1,$$

$$p_1 = \frac{2\Gamma(m/2+1)}{\pi^{m/2}} \cdot \frac{1}{B'^m + B_0^m}.$$

From the definition of rejection sampling, we know that the expected number M_{reject} of repetitions is:

$$M_{\mathsf{reject}} = \max \frac{f(x)}{p(x) \cdot g(x)} = \frac{2 \cdot B^m}{B'^m + B_0^m}. \tag{5}$$

Expected Number M_{comp} of Repetitions in Hint Generation Subsequently, we analyze the impact of the rejection sampling produced within hint generation.

Step 9 of the signing algorithm may fail for both $b = 0$ and $b = 1$. The two cases differ only in sign and can be analyzed in the same way. Therefore, we focus on the case $b = 0$, while the case $b = 1$ follows analogously. Due to the particular structure of the vector α, if the equality does not hold, it can only be the first polynomial that differs. Thus, we only need to consider $w_0 \in R_q$, and its explicit form can be written as:

$$w_0 = a_0 + a_1 x + ... + a_{n-1} x^{n-1}.$$

Similarly, the challenge $c \in R_2$ can be expressed as:

$$c = c_0 + c_1 x + ... + c_{n-1} x^{n-1}.$$

Since c is sparse with only τ positions equal to 1, w_0 differs from $w_0 + c$ in at most τ coefficient positions. Next, we analyze the probability that two coefficients, which differ at the same positions, remain unequal after compression. Specifically, our goal is to determine the probability that $\mathsf{Comp}_d(a) \neq \mathsf{Comp}_d(a+1)$. According to the definition of the compression method, this probability is easily obtained as:

$$p_3 = \frac{2^d}{q}.$$

Therefore, the acceptance probability for this part of the rejection sampling is:

$$p_{\mathsf{comp}} = (\frac{q - 2^d}{q})^\tau.$$

Consequently, the expected number of repetitions required for this part can be computed as:

$$M_{\mathsf{comp}} = \frac{1}{p_{\mathsf{comp}}}. \tag{6}$$

Expected Number M of Repetitions in Signing

$$M = M_{\mathsf{reject}} \times M_{\mathsf{comp}}.$$

1.5 Parameter sets

We define three parameter sets for our scheme and denote them by DARTS-128, DARTS-256, DARTS-512 in Table 2. This table presents the number of sampling rejections, public key size, secret key size, signature size, and total size under different parameter choices. The core objective of parameter selection is to achieve optimal operational efficiency and communication overhead while ensuring rigorous security.

The overall signing efficiency is characterized by the expected repetition count M. This is the product of the rejection sampling expectation M_{reject}, and M_{comp}. The final communication overhead is measured by the public key size and signature size in bytes.

Table 2: Parameter sets under different security level.

DARTS				
Security level		**128 bits**	**256 bits**	**512 bits**
n	Degree of $\mathcal{R}$	512	512	1024
(k, ℓ)	Dimensions of $\mathbf{z_2}, \mathbf{z_1}$	(1,2)	(2,3)	(2,3)
q	Modulus	130817	130817	260609
$\rho(\lambda)$	Byte length of the random seed	32	32	64
τ	Weight of c	30	59	115
p	$\mathbf{s}$ distribution parameter	0.15	0.3125	0.20
γ	**sk** rejection parameter	34.92	62.02	72.80
$\lfloor \log_2 \lvert C \rvert \rfloor$	$\lfloor \log_2 \binom{n}{\tau} \rfloor$	161	257	513
$B_{\mathbf{s}c}$	Upper-bound of $\mathbf{s}c$	191.26	476.38	780.72
d	Compression parameter	10	9	10
Rejection Sampling Parameters				
B''	Verify radius	7500.00	23000.00	80000.00
B_0	Rejection radius	5866.64	18440.32	73440.24
B'	Rejection radius	6054.88	18910.70	74216.86
B	y radius	6057.90	18916.70	74220.97
B_{SIS}	SIS radius	10435.93	31240.06	91607.91
Expected Number of Repetitions				
sk acceptance rate		0.5	0.5	0.5
M_{reject}	Rejection sampling	4.30	4.50	2.65
M_{comp}	Compression	1.27	1.26	1.57
M	$M_{\mathsf{reject}} \times M_{\mathsf{comp}}$	5.44	5.67	4.17
Public Key Size and Signature Size				
Public key size (bytes)		1120	2208	4672
Signature size (bytes)		1449	2489	5852
Sum size: (bytes)		2569	4697	10524
Secret key size (bytes)		1536	2880	6016

2 Design Rationale

The Fiat-Shamir with Aborts paradigm, introduced by Lyubashevsky [16], converts SIS/LWE-based identification protocols into non-interactive signatures. It uses rejection sampling to prevent the leakage of sensitive information. In the signing procedure, the signer first samples a random masking vector $\mathbf{y}$, computes a challenge $c = \mathsf{H}(\mathsf{h}(\mathbf{y}), M)$, and forms $\mathbf{z} = \mathbf{y} + \mathbf{s}c$. A rejection condition $p(\mathbf{z})$ is then checked; if it fails, the process aborts and restarts. The verifier then accepts the signature only if $\mathbf{z}$ is sufficiently short and the reproduced challenge c' is identical to original challenge c. This mechanism ensures the output distribution of $\mathbf{z}$ is independent of the secret key $\mathbf{s}$, preventing sensitive information leakage. Within this framework, Ducas et al. [11] first introduced bimodal rejection sampling to reduce rejection rate, yet its reliance on discrete Gaussian distributions posed side-channel risks due to the difficulty of constant-time implementation. Cheon et al. [8] resolves this by adopting a uniform distribution over hyperball, ensuring security against timing and power attacks. Then Woo et al. [20] proposed a scheme which is designed based on a novel NTRU-based key structure that fits well with bimodal gaussian distributions.

Our signature scheme is designed following the basic framework above. More precisely:

- Within the Fiat-Shamir with aborts paradigm, we adopt bimodal rejection sampling over hyperballs using the ℓ_2-norm, which can achieve a lower rejection rate than its unimodal counterpart. The distribution of the signature is set to be the double sphere distribution to further decrease the rejection rate.

- We employ the MNLWE problem to ensure the security of private key and the BimodalSelfTargetMSIS problem to ensure the security of signature. See Subsection 3.1 for the definitions of both the hard problems.

  The use of MNLWE: We employ a new hardness assumption, MNLWE, similar to those problems used in [1, 2, 14], and provide a reduction from the standard MLWE problem. Specifically, this new assumption ensures the key-recovery security by guaranteeing that the public key is computationally indistinguishable from a uniform distribution. Furthermore, utilizing this assumption for key generation, DARTS can achieve higher security strength in practice than the standard MLWE under the same parameter sets. Consequently, we can employ a more sparse secret distribution that yields smaller offset centers $\mathbf{s}c$ to reduce rejection rate, further enhancing the overall performance.

  The use of BimodalSelfTargetMSIS: To match the bimodal rejection sampling, we employ the BimodalSelfTargetMSIS assumption which can reduce from MSIS for forgery security. Hence, for the security analysis of BimodalSelfTargetMSIS, we analyze the hardness of the corresponding MSIS.

- Parameter selection. Our parameter selection follows one core goal: minimizing signature size and improving efficiency while maintaining high security.

  First, we select parameters — including dimensions (n, k, l), modulus q, and the ℓ_2-norm bound B'' for verification, among others — to meet different security levels. The security strength is mainly evaluated via the Core-SVP model. See Subsection 3.4 for more details.

  After meeting these security needs, we optimize parameters to reduce the total signature size and enhance efficiency. Specifically, using a sparse secret allows for smaller offset centers in our bimodal sampling, leading to smaller signature components. To further enhance efficiency, we choose specific values for n and q that support the Number Theoretic Transform (NTT) for fast polynomial multiplication.

- Improving efficiency. We lower the rejecting rate by

  - decreasing the center offset $\mathbf{s}c$: a more sparse $\mathbf{s}$ together with an additional rejection sampling step for the secret $\mathbf{s}$ during key generation to bound the center offset $\mathbf{s}c$.
  - choosing a new distribution for the signature.

- Compactness enhancement. To significantly reduce signature size with modest efficiency overhead, our scheme utilizes a novel bimodal rejection sampling over hyperballs which constructs a smaller hyperball within the intersection of two offset hyperballs. Hence, it increases the probability of generating small-sized signatures. Due to the spherical symmetry, this process ensures that the distribution of the output signatures does not reveal information about the secret $\mathbf{s}$. To further enhance signature compactness, we construct a compact hint $\mathbf{h}$ by using $\mathbf{z}_2$ and ω. Hence, we can eliminate the need for explicit transmission of raw $\mathbf{z}_2$. Additionally, we perform modulo switching [5] on $\mathbf{z}_1$ to scale down its coefficient from size $\lceil \log_2 q \rceil$ to size d. The compressed signature $\mathsf{Comp}_d(\mathbf{z}_1)$ together with the compact hint $\mathbf{h}$ are subsequently encoded with rANS encoder. This multi-layer compression design is critical for constraining the overall communication cost and achieving optimal signature compression performance.

3 Security Analysis

This section presents the security analysis of DARTS. First, we define the underlying hardness assumptions and establish firm security reductions from these classical problems to guarantee provable security. Second, we introduce the formal security definitions considered in this work. Finally, we analyze the security properties of the underlying Canonical Identification scheme(CID) and evaluate the concrete security levels against various attacks.

3.1 Hardness assumption

The MLWE Problem. For positive integers q, k, l and a probability distribution S_η, we say that the advantage of algorithm $\mathcal{A}$ in solving the decisional $\text{MLWE}_{q,k,l,\eta}$ problem over the ring $\mathcal{R}_q$ is

$$\begin{aligned}\text{Adv}^{\text{MLWE}}_{q,k,l,\eta}(\mathcal{A}) = \big|&\Pr[b = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_q^{k\times l}, \mathbf{u} \leftarrow \mathcal{R}_q^k; b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{u})] \\ &- \Pr[b = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_q^{k\times l}, (\mathbf{s}_1, \mathbf{e}_1) \leftarrow S_\eta^{l+k}; b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{A}\mathbf{s}_1 + \mathbf{e}_1)]\big| .\end{aligned}$$

The MSIS Problem. For positive integers q, k, l and a bound $\beta > 0$, we say that the advantage of algorithm $\mathcal{A}$ in solving the search $\text{MSIS}_{q,k,l,\beta}$ problem over the ring $\mathcal{R}_q$ is

$$\text{Adv}^{\text{MSIS}}_{q,k,l,\beta}(\mathcal{A}) = \Pr[0 \le \|y\|_2 \le \beta \wedge (\mathbf{A}, \mathbf{I}) \cdot y = 0 \mid \mathbf{A} \leftarrow \mathcal{R}_q^{k\times l}; y \leftarrow \mathcal{A}(\mathbf{A})].$$

The BimodalSelfTargetMSIS Problem Let $\mathsf{H} : \{0,1\}^* \times M \to \mathcal{R}_2$ be a cryptographic hash function, where $M \subseteq \{0,1\}^*$ is the message space. For positive integers q, k, l and a bound $\beta > 0$, the $\text{BimodalSelfTargetMSIS}_{\mathsf{H},q,k,l,\beta}$ problem is defined as follows.

An adversary $\mathcal{A}$ is given a matrix $\mathbf{A}$ generated by sampling $(\mathbf{A}_0 \mid \mathbf{A}_1) \leftarrow \mathcal{R}_q^{k\times l}$ uniformly at random and setting $\mathbf{A} = (\mathbf{A}_0 \mid \mathbf{A}_1 \mid \mathbf{I}_k) \bmod q$. The adversary is also given oracle access to the hash function H.

The adversary $\mathcal{A}$ outputs a tuple $(\mathbf{z}, c, \mu)$, where $\mathbf{z} \in \mathcal{R}^{k+l}$, $c \in \mathcal{R}_2$, and $\mu \in M$. We say that $\mathcal{A}$ succeeds if

$$0 < \|\mathbf{z}\|_2 < \beta \quad \text{and} \quad \mathsf{H}\left(\mathbf{A}\mathbf{z} + \frac{q+1}{2} \cdot c \cdot \alpha \bmod q,\ \mu\right) = c,$$

where $\alpha \in \mathcal{R}_2^{k+l}$ is a fixed public vector associated with the bimodal construction.

The advantage of $\mathcal{A}$ in solving $\text{BimodalSelfTargetMSIS}_{\mathsf{H},n,q,k,l,\beta}$ is defined as

$$\text{Adv}^{\text{BimodalSelfTargetMSIS}}_{\mathsf{H},q,k,l,\beta}(\mathcal{A}) = \Pr[\mathcal{A} \text{ succeeds}] .$$

The MNLWE Problem For positive integers q, k, l, the $\text{MNLWE}_{q,k,l}$ problem is defined as follows.

An adversary $\mathcal{A}$ is given an instance $(\mathbf{A}_0, \mathbf{A}_1, \frac{q+1}{2}\alpha) \in \mathcal{R}_q^k \times \mathcal{R}_q^{k\times(l-1)} \times \mathcal{R}_q^k$ distributed according to one of the following two cases:

- Uniform case: $(\mathbf{A}_0, \mathbf{A}_1) \leftarrow \mathcal{R}_q^{k\times l}$ uniformly at random and get the instance $(\mathbf{A}_0, \mathbf{A}_1, \frac{q+1}{2}\alpha)$;
- MNLWE case: Generate an MLWE instance $(\mathbf{A}_1, \mathbf{b}) \in R_q^{k\times(l-1)} \times R_q^k$, where $\mathbf{b} \equiv \mathbf{A}_1\mathbf{s}_1 + \mathbf{e}_1 \pmod q$ for a secret-error tuple $(\mathbf{s}_1, \mathbf{e}_1) \in S_\eta^{l+k-1}$. Then for a randomly chosen invertible $s_0 \in S_\eta$, compute $\mathbf{A}_0 \equiv -s_0^{-1}(\mathbf{b} - \frac{q+1}{2} \cdot \alpha) \pmod q$ to get the instance $(\mathbf{A}_0, \mathbf{A}_1, \frac{q+1}{2}\alpha)$.

The advantage of $\mathcal{A}$ in solving Decision-$\text{MNLWE}_{q,k,l,\eta}$ is defined as

$$\begin{aligned}\text{Adv}^{\text{MNLWE}}_{n,q,k,l,\eta}(\mathcal{A}) = \Big|&\Pr[b = 1 \mid (\mathbf{A}_1, \mathbf{A}_0) \leftarrow \mathcal{R}_q^{k\times l}; b \leftarrow \mathcal{A}(\mathbf{A}_0, \mathbf{A}_1, \frac{q+1}{2}\alpha)] \\ &- \Pr[b = 1 \mid \mathbf{A}_1 \leftarrow \mathcal{R}_q^{k\times(l-1)}, \mathbf{A}_0 \leftarrow (-s_0^{-1}(\mathbf{b} - \frac{q+1}{2} \cdot \alpha), \\ &b \leftarrow \mathcal{A}(\mathbf{A}_0, \mathbf{A}_1, \frac{q+1}{2}\alpha)]\Big| .\end{aligned}$$

Having defined the standard lattice problems, we first establish the hardness of the two variants used in our construction through formal reductions. Specifically, we show that MSIS reduces to BimodalSelfTargetMSIS and that MLWE reduces to MNLWE. We then complement these theoretical results with a concrete security analysis of MNLWE against the best-known primal and dual lattice attacks, following the framework used for NTWE [14].

Theorem 2. Let $q > 0$ be an odd modulus, $\text{H} : \{0,1\}^* \times M \to \mathcal{R}_2$ be a hash function modeled as a random oracle in cryptography, and assume that every polynomial-time classical algorithms have negligible advantage against the $\text{MSIS}_{n,q,k,l,4\beta+\sqrt{2\tau}}$ problem. Then, every polynomial-time classical algorithm also have negligible advantage against the $\text{BimodalSelfTargetMSIS}_{n,q,k,l,\beta}$ problem.

Proof. Consider a classical algorithm $\mathcal{A}$ against the BimodalSelfTargetMSIS$_{n,q,k,l,\beta}$ problem that runs in polynomial time and has classical access to H. If $\mathcal{A}^{\mathsf{H}(\cdot)}$ makes Q hash queries $\mathsf{H}(w_i, \mu_i)$ (for $i = 1, \cdots, Q$) and outputs a solution $(\mathbf{z}, c, \mu_l)$ for some $l \in [Q]$, then we can construct an adversary $\mathcal{A}'$ against the MSIS$_{n,q,k,l,4\beta+\sqrt{\tau}}$ problem as follows.

Adversary $\mathcal{A}'$ can first rewind $\mathcal{A}$ to the point of making the l-th query and reprogram the hash value to $\mathsf{H}(w_l, \mu_l) = c'(\neq c)$. Then, with probability approximately $1/Q$, algorithm $\mathcal{A}$ will produce another solution $(\mathbf{z}', c', \mu_l)$. We thus have

$$\begin{cases} \mathbf{A}\mathbf{z} + \frac{q+1}{2} c \cdot \alpha \equiv w_l \equiv \mathbf{A}\mathbf{z}' + \frac{q+1}{2} c' \cdot \alpha \mod q, \\ \|\mathbf{z}\|_2, \|\mathbf{z}'\|_2 < \beta. \end{cases}$$

From the above equations,

$$\mathbf{A}(\mathbf{z} - \mathbf{z}') \equiv \frac{q+1}{2}(c' - c) \cdot \alpha \pmod q.$$

Multiplying both sides by 2 yields

$$2\mathbf{A}(\mathbf{z} - \mathbf{z}') \equiv (c' - c) \cdot \alpha \pmod q.$$

Rearranging gives

$$\mathbf{A} \cdot \begin{bmatrix} 2\mathbf{z}_u - 2\mathbf{z}'_u \\ 2\mathbf{z}_m - 2\mathbf{z}'_m + c - c' \\ 2\mathbf{z}_d - 2\mathbf{z}'_d \end{bmatrix} \equiv 0 \pmod q.$$

where $\mathbf{z} = \begin{pmatrix} \mathbf{z}_u \\ \mathbf{z}_m \\ \mathbf{z}_d \end{pmatrix}$ is a concatenated vector with sub-vector lengths of $l, 1$, and $k-1$, respectively. Since there exists a coefficient of $c - c'$ that is $1 \pmod 2$,

$$2\mathbf{z}_m - 2\mathbf{z}'_m + c - c' \neq 0,$$

and

$$\|(2\mathbf{z}_u - 2\mathbf{z}'_u, 2\mathbf{z}_m - 2\mathbf{z}'_m + c - c', 2\mathbf{z}_d - 2\mathbf{z}'_d)\|_2 \leq 2\|\mathbf{z} - \mathbf{z}'\|_2 + \|(0, c - c', 0)\|_2 \leq 4\beta + \sqrt{2\tau}.$$

Therefore, this provides a solution $(2\mathbf{z}_u - 2\mathbf{z}'_u, 2\mathbf{z}_m - 2\mathbf{z}'_m + c - c', 2\mathbf{z}_d - 2\mathbf{z}'_d)$ to the MSIS$_{n,q,k,l,4\beta+\sqrt{2\tau}}$ problem for the matrix $(\mathbf{A}_0, \mathbf{A}_1)$.

□

Theorem 3 (Reduction from MLWE to MNLWE). Let $q, k, l, \eta \in \mathbb{Z}_{>0}$. For every polynomial-time algorithms $\mathcal{A}$ against the MNLWE$_{q,k,l,\eta}$ problem, there exists a polynomial-time algorithm $\mathcal{B}$ such that

$$\mathrm{Adv}^{\mathrm{MNLWE}}_{q,k,l,\eta}(\mathcal{A}) \leq \mathrm{Adv}^{\mathrm{MLWE}}_{q,k,l-1,\eta}(\mathcal{B}).$$

Proof. On input $(\mathbf{A}_1, \mathbf{b}) \in R_q^{k \times (l-1)} \times R_q^k$, the algorithm $\mathcal{B}$ first samples invertible $s_0 \leftarrow S_\eta$ and computes $\mathbf{A}_0 := -s_0^{-1}(\mathbf{b} - \frac{q+1}{2}\alpha)$, then output $\mathcal{A}(\mathbf{A}_0, \mathbf{A}_1, \frac{q+1}{2}\alpha)$.

Case 1: Uniform case. If $\mathbf{b} \leftarrow R_q^k$ is uniform, then $\mathbf{b} - \frac{q+1}{2}\alpha$ is also uniform. Since multiplication by the invertible element s_0^{-1} is a bijection over R_q, we have $\mathbf{A}_0 \leftarrow R_q^k$ uniformly. Thus, the tuple $(\mathbf{A}_0, \mathbf{A}_1, \frac{q+1}{2}\alpha)$ is distributed as in the uniform case of MNLWE (where the target α is a fixed public constant).

Case 2: MLWE Case. In this case, $\mathbf{A}_0 = -s_0^{-1}(\mathbf{A}_1\mathbf{s}_1 + \mathbf{e}_1 - \frac{q+1}{2}\alpha)$. Let $\mathbf{s}' = (s_0, \mathbf{s}_1)^\top \in R_q^l$. We observe that $(\mathbf{A}_0, \mathbf{A}_1)\mathbf{s}' = \mathbf{A}_0 s_0 + \mathbf{A}_1\mathbf{s}_1 = -(\mathbf{b} - \frac{q+1}{2}\alpha) + \mathbf{A}_1\mathbf{s}_1 = -(\mathbf{A}_1\mathbf{s}_1 + \mathbf{e}_1 - \frac{q+1}{2}\alpha) + \mathbf{A}_1\mathbf{s}_1 = \frac{q+1}{2}\alpha - \mathbf{e}_1$. Rearranging this yields $\frac{q+1}{2}\alpha = (\mathbf{A}_0, \mathbf{A}_1)\mathbf{s}' + \mathbf{e}$, where $\mathbf{e}_1$ is the error vector. This matches the MNLWE distribution exactly. □

These reductions establish the theoretical hardness of the two modified problems. We next assess the concrete security of MNLWE under the selected parameters by considering the best-known lattice attacks.

Primal Attack The primal attack against MNLWE is based on the lattice construction used for the primal attack against MLWE. More precisely, consider an MNLWE instance

$$(\mathbf{A}_1, \mathbf{A}_0 = -s_0^{-1}(\mathbf{A}_1\mathbf{s}_1 + \mathbf{e} - \frac{q+1}{2}\alpha)) \in \mathcal{R}_q^{k\times(l-1)} \times \mathcal{R}_q^k.$$

The attack aims to either distinguish the given samples from uniformly random ones or recover the secret components. Following the standard primal attack framework for MLWE, we reduce the MNLWE problem to finding a sufficiently short vector in the lattice generated by the columns of the following basis.

$$\begin{pmatrix} q\mathbf{I}_k & 0 & 0 & 0 \\ \mathbf{A}_1^\top & \mathbf{I}_l & 0 & 0 \\ \mathbf{A}_0^\top & 0 & \mathbf{I} & 0 \\ \frac{q+1}{2}\alpha & 0 & 0 & 1 \end{pmatrix}$$

where $\mathbf{A}_1 \in \mathbb{Z}^{nk\times n(l-1)}$, $\mathbf{A}_0 \in \mathbb{Z}^{nk\times n}$.

This MNLWE lattice contain the secret vector

$$(\mathbf{k}^\top, \mathbf{s}_1^\top, s_0, -1) \cdot \begin{pmatrix} q\mathbf{I}_k & 0 & 0 & 0 \\ \mathbf{A}_1^\top & \mathbf{I}_l & 0 & 0 \\ \mathbf{A}_0^\top & 0 & \mathbf{I} & 0 \\ \frac{q+1}{2}\alpha & 0 & 0 & 1 \end{pmatrix} = (-\mathbf{e}^\top, \mathbf{s}_1^\top, s_0, -1).$$

To further understand the security of MNLWE, we compare its lattice structure with the one arising from MLWE. The primal lattice for rank-l MNLWE follows a construction similar to that of the primal lattice for rank-l MLWE. Let $\mathbf{H} = (\mathbf{A}_1, \mathbf{A}_0) \in \mathcal{R}_q^{k\times l}$, the lattice is given by

$$\begin{pmatrix} q\mathbf{I}_k & 0 & 0 \\ \mathbf{H}^\top & \mathbf{I} & 0 \\ \frac{q+1}{2}\alpha & 0 & 1 \end{pmatrix}$$

The resulting lattice has the same dimension as the corresponding lattice for a rank-l MLWE problem. However, the construction of the $\mathbf{H}$ matrix differs due to the specific structure of MNLWE. Moreover, for the same number of samples, the resulting lattice has the same determinant as the corresponding MLWE lattice, and the associated target vector has a comparable length.

The above comparison shows that the primal lattice for MNLWE has a structure close to that of MLWE. Although MNLWE introduces an additional algebraic relation in the matrix generation, the key lattice parameters, such as dimension, determinant, and target vector length, remain similar. Hence, standard lattice hardness estimates suggest that $\text{MNLWE}_{q,k,l,\eta}$ and the corresponding $\text{MLWE}_{q,k,l,\eta}$ instance have comparable primal attack costs. We use the latter as a reference for estimating the concrete security of $\text{MNLWE}_{q,k,l,\eta}$.

Dual Attack. We next consider a dual lattice attack against MNLWE. The attack follows the general idea of the dual attack against MLWE, but the algebraic relation in an MNLWE instance leads to an additional NTRU-type lattice problem. Given an MNLWE instance

$$(\mathbf{A}_1, \mathbf{A}_0) \in \mathcal{R}_q^{k\times(l-1)} \times \mathcal{R}_q^k,$$

where

$$\mathbf{A}_0 = -s_0^{-1}\left(\mathbf{A}_1\mathbf{s}_1 + \mathbf{e} - \frac{q+1}{2}\boldsymbol{\alpha}\right) \pmod q. \tag{7}$$

Here, $\mathbf{s}_1 \in \mathcal{R}_q^{l-1}$, $s_0 \in \mathcal{R}_q$, and $\mathbf{e} \in \mathcal{R}_q^k$ are short.

We consider the dual lattice

$$\mathcal{L}^\perp(\mathbf{A}_1) = \left\{(\mathbf{x}, \mathbf{y}) : \mathbf{x}^\mathsf{T}\mathbf{A}_1 = \mathbf{y}^\mathsf{T} \pmod q\right\},$$

where $\mathbf{x} \in \mathcal{R}^k$ and $\mathbf{y} \in \mathcal{R}^{l-1}$. After coefficient embedding, this lattice has dimension $(k+l-1)n$ and determinant $q^{(l-1)n}$

Let

$$\mathbf{w} = (\mathbf{x}, \mathbf{y}) \in \mathcal{L}^\perp(\mathbf{A}_1)$$

be a short vector and denote $W = \|\mathbf{w}\|$. By the definition of the dual lattice, $\mathbf{x}^\mathsf{T}\mathbf{A}_1 = \mathbf{y}^\mathsf{T} \pmod q$. Multiplying (7) by $\mathbf{x}^\mathsf{T}$ therefore gives

$$\begin{aligned}\mathbf{x}^\mathsf{T}\mathbf{A}_0 &= -s_0^{-1}\left(\mathbf{x}^\mathsf{T}\mathbf{A}_1\mathbf{s}_1 + \mathbf{x}^\mathsf{T}\mathbf{e} - \frac{q+1}{2}\mathbf{x}^\mathsf{T}\boldsymbol{\alpha}\right) \pmod q \\ &= -s_0^{-1}\left(\mathbf{y}^\mathsf{T}\mathbf{s}_1 + \mathbf{x}^\mathsf{T}\mathbf{e} - \frac{q+1}{2}\mathbf{x}^\mathsf{T}\boldsymbol{\alpha}\right) \pmod q. \end{aligned} \tag{8}$$

Multiplying (8) by 2, we obtain

$$2\mathbf{x}^\mathsf{T}\mathbf{A}_0 = -s_0^{-1}\left(2\mathbf{y}^\mathsf{T}\mathbf{s}_1 + 2\mathbf{x}^\mathsf{T}\mathbf{e} - \mathbf{x}^\mathsf{T}\boldsymbol{\alpha}\right) \pmod q.$$

Define $g = -\left(2\mathbf{y}^\mathsf{T}\mathbf{s}_1 + 2\mathbf{x}^\mathsf{T}\mathbf{e} - \mathbf{x}^\mathsf{T}\boldsymbol{\alpha}\right)$ and $h = 2\mathbf{x}^\mathsf{T}\mathbf{A}_0$. Then

$$h = gs_0^{-1} \pmod q. \tag{9}$$

Hence, a short vector in $\mathcal{L}^\perp(\mathbf{A}_1)$ transforms the MNLWE instance into an NTRU-type instance whose short secret pair is (g, s_0).

We next estimate the norm of g. Following the usual coefficient-norm heuristic for products of short elements in power-of-two cyclotomic rings, Ignoring the lower-order contribution of the term involving $\boldsymbol{\alpha}$, we have the norm

$$\|g\|_2 \approx 2W\sqrt{2p(k+l-1)n},$$

where p is the probability of 1(-1) for each coefficient of $\mathbf{s}$.

The NTRU-type instance (9) corresponds to the $2n$-dimensional lattice generated by

$$\mathbf{B}_h = \begin{pmatrix} q\mathbf{I} & \mathbf{H} \\ 0 & \mathbf{I} \end{pmatrix},$$

where $\mathbf{H}$ is the integer matrix corresponding to multiplication by h. This lattice contains the short vector (g, s_0). Its determinant is q^n. By the Gaussian heuristic, a random $2n$-dimensional lattice with determinant q^n is expected to have a shortest vector of length approximately $\sqrt{\frac{nq}{\pi e}}$. Since $\|s_0\|_2 \approx \sqrt{2pn}$, the constructed NTRU-type lattice contains an unusually short vector only if

$$||g||_2^2 + 2pn \lesssim \frac{nq}{\pi e}.$$

Therefore, a short vector in the dual lattice transforms the MNLWE instance into an NTRU-type relation involving two short elements (g, s_0). The resulting NTRU-type lattice contains the short vector (g, s_0). As in the dual attack against NTWE [14], this lattice can be rebalanced by appropriately scaling the two components, after which the cost of recovering (g, s_0) can be estimated using standard lattice-reduction techniques. We omit the analogous analysis here.

Concrete Security Estimation. Based on the above lattice analyses, we estimate the concrete security of MNLWE with respect to the best-known lattice attacks. The primal lattice of MNLWE has comparable parameters to the corresponding rank-l MLWE lattice. Therefore, we use the rank-l MLWE primal attack as a reference for estimating the cost of the primal attack against MNLWE. For the dual attack, although the first lattice-reduction step is related to the dual lattice of a rank-$(l-1)$ MLWE instance, an additional NTRU-type lattice reduction step is required to recover the short pair (g, s_0). Therefore, the dimension loss in the MLWE reduction does not directly translate into a practical attack with the same cost as the corresponding lower-rank MLWE instance. In our parameter selection, we use the rank-l MLWE primal attack cost as the reference security estimate for MNLWE.

3.2 Security definition

Definition 3 (UF-CMA Security)**.** Let λ be the security parameter. A digital signature scheme $\mathsf{Sig} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ is said to be existentially unforgeable under adaptive chosen-message attacks (UF-CMA) if for all probabilistic polynomial-time adversaries $\mathcal{A}$,

$$\Pr\left[\begin{array}{ll} (\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda); & \mathsf{Verify}(\mathsf{pk}, m^\star, \sigma^\star) = 1 \\ (m^\star, \sigma^\star) \leftarrow \mathcal{A}^{\mathsf{Sign}(\mathsf{sk},\cdot)}(\mathsf{pk}) \quad : & \wedge\ m^\star \notin \mathcal{Q} \end{array}\right] \leq \mathsf{negl}(\lambda),$$

where $\mathcal{Q}$ denotes the set of messages queried by $\mathcal{A}$ to the signing oracle.

UF-NMA (Unforgeability under No-Message Attack) is defined as UF-CMA, except that the adversary is not given access to the signing oracle and only receives the public key.

SUF-CMA (Strong Unforgeability under Chosen-Message Attacks) is defined as UF-CMA, except that a successful forgery consists of any fresh valid message–signature pair $(m^\star, \sigma^\star)$, including a new signature on a previously queried message.

Definition 4 (Computational Unique Response (CUR))**.** Let λ be the security parameter. A digital signature scheme $\mathsf{Sig} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ is said to satisfy the computational unique response (CUR) property if for all probabilistic polynomial-time adversaries $\mathcal{A}$,

$$\Pr\left[\begin{array}{l|ll} & & \mathsf{Verify}(\mathsf{pk}, m, \sigma_1) = 1 \\ (\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda); & : & \wedge\ \mathsf{Verify}(\mathsf{pk}, m, \sigma_2) = 1 \\ (m, \sigma_1, \sigma_2) \leftarrow \mathcal{A}(\mathsf{pk}) & & \wedge\ \sigma_1 \neq \sigma_2 \end{array}\right] \leq \mathsf{negl}(\lambda).$$

Definition 5 (Statistical Accept Honest-Verifier Zero-Knowledge (saHVZK))**.** Let λ be the security parameter. A canonical identification scheme ID = (Gen,$\mathcal{P}$,$\mathcal{V}$) is said to satisfy statistical accept honest-verifier zero-knowledge (saHVZK) if there exists a PPT simulator Sim such that for all $(pk, sk) \leftarrow \mathrm{Gen}(1^\lambda)$,

$$\Delta(\{(w, c, z) \leftarrow \langle \mathcal{P}(sk), \mathcal{V}(pk)\rangle \text{ s.t. } z \neq \perp\}, \{(w', c', z') \leftarrow \mathrm{Sim}(pk)\}) \leq \mathrm{negl}(\lambda).$$

3.3 Security reduction

The Fiat-Shamir transform converts an identification scheme into a digital signature. Consequently, proving the security of the signature scheme requires the underlying CID to satisfy several key properties. We first establish these properties for our CID; we then provide the formal security reduction for the resulting signature scheme.

3.3.1 Security Properties of the Underlying CID

CID scheme

$\underline{\mathrm{P}(\mathbf{A}, \mathbf{s})}$		$\underline{\mathrm{V}(\mathbf{A})}$
$(\mathbf{y}_1, \mathbf{y}_2) \leftarrow \mathcal{B}_{(1/N)\mathbb{R}, l+k}(B)$		
$\omega = \mathbf{A}\lfloor \mathbf{y} \rceil \bmod q$		
$\omega_1 = \mathsf{Comp}_d(w)$		
	$\xrightarrow{\omega_1}$	$c \leftarrow \mathcal{U}(\mathcal{C})$
$b \leftarrow \mathsf{U}(\{0,1\})$		
	$\xleftarrow{c}$	
$(\mathbf{z}_1, \mathbf{z}_2) = (\mathbf{y}_1, \mathbf{y}_2) + (-1)^b \mathbf{s} c$		
If $p(\mathbf{z})$, restart		
Else $\sigma = \mathsf{Compress}(\lfloor \mathbf{z} \rceil, c)$		
	$\xrightarrow{\sigma}$	
		$\tilde{\mathbf{z}} = \mathsf{Decompress}(\mathbf{A}, \sigma, c)$
		Accept if $\|\tilde{\mathbf{z}}\|_2 \leq B''$ and recover $c' = c$

Statistical Accept Honest-Verifier Zero-Knowledge. In this section, we show that the DARTS identification scheme with the modified compression-stability condition

$$\mathsf{Comp}_d(\omega) = \mathsf{Comp}_d\big(\omega + (1-2b)\alpha c\big)$$

achieves statistical honest-verifier zero-knowledge for accepting executions. More precisely, we show that the distribution of accepting transcripts generated by the real protocol is statistically close to that generated by the CID simulator. If the rejection-sampling procedure produces its target distribution exactly, then the two distributions are, in fact, identical.

CID simulator

1. $(\mathbf{z}_1, \mathbf{z}_2) \leftarrow \mathcal{B}_{\mathcal{B}}(B', B_0)^{l+k}$
2. $c \leftarrow \mathcal{U}(\mathcal{C})$
3. $\mathbf{u} = \mathbf{A}\,\lfloor \mathbf{z} \rceil + \frac{q+1}{2}\alpha c$

If condition $\mathsf{Comp}_d(\mathbf{u}) = \mathsf{Comp}_d(\mathbf{u} - \alpha c)$ **:**

4. $\omega_1 = \mathsf{Comp}_d(u)$
5. $\mathbf{h} = \omega_1 - \mathsf{Comp}_d(\bar{\mathbf{A}}\,\lfloor \mathbf{z}_1 \rceil + \frac{q+1}{2}\alpha c)$
6. $\mathbf{x} = \mathsf{Encode}(\mathsf{Comp}_d(\lfloor \mathbf{z}_1 \rceil))$
7. $\mathbf{e}_z = \lfloor \mathbf{z}_1 \rceil - \mathsf{Decomp}_d(\mathsf{Comp}_d(\lfloor \mathbf{z}_1 \rceil))$

8. **return** $(\mathbf{w} = \omega_1, \sigma = (c, \mathbf{x}, \mathbf{e}_z, \mathbf{h}))$

Let $\mathcal{D}_{\mathbf{z}}$ denote the target distribution of the response vector $\mathbf{z} = (\mathbf{z}_1, \mathbf{z}_2)$. By the rejection-sampling property, before applying the compression-stability condition, the joint distribution of $(\mathbf{z}, c, b)$ in the real protocol is statistically close to

$$\mathcal{D}_{\mathbf{z}} \times U(\mathcal{C}) \times U(\{0,1\}).$$

Let $\varepsilon_{\mathsf{RS}}$ denote the corresponding statistical distance. In particular, conditioned on either value of b, the distribution of $\mathbf{z}$ is independent of the secret key and statistically close to $\mathcal{D}_{\mathbf{z}}$.

For a response $\mathbf{z}$ and challenge c, define

$$\mathbf{u} := \mathbf{A}\lfloor \mathbf{z} \rceil + \frac{q+1}{2}\alpha c \pmod q.$$

Using

$$\lfloor \mathbf{z} \rceil = \lfloor \mathbf{y} \rceil + (-1)^b \mathbf{s} c, \qquad \omega = \mathbf{A}\lfloor \mathbf{y} \rceil,$$

and the public-key relation

$$\mathbf{A}\mathbf{s} = \frac{q+1}{2}\alpha \pmod q,$$

we obtain

$$\begin{aligned}
\mathbf{u} &= \mathbf{A}\big(\lfloor \mathbf{y} \rceil + (-1)^b \mathbf{s} c\big) + \frac{q+1}{2}\alpha c \\
&= \omega + \big(1 + (-1)^b\big)\frac{q+1}{2}\alpha c \pmod q \\
&= \begin{cases} \omega + \alpha c \pmod q, & b = 0, \\ \omega \pmod q, & b = 1, \end{cases}
\end{aligned}$$

where we used

$$(q+1)\alpha c \equiv \alpha c \pmod q.$$

We now rewrite the modified compression-stability condition in terms of $(\mathbf{z}, c)$. When $b = 0$, we have $\omega = \mathbf{u} - \alpha c$, and hence

$$\mathsf{Comp}_d(\omega) = \mathsf{Comp}_d(\omega + \alpha c)$$

is equivalent to

$$\mathsf{Comp}_d(\mathbf{u} - \alpha c) = \mathsf{Comp}_d(\mathbf{u}).$$

When $b = 1$, we have $\omega = \mathbf{u}$, and the condition

$$\mathsf{Comp}_d(\omega) = \mathsf{Comp}_d(\omega - \alpha c)$$

is again equivalent to

$$\mathsf{Comp}_d(\mathbf{u}) = \mathsf{Comp}_d(\mathbf{u} - \alpha c).$$

Therefore, in both branches, the compression-stability test is precisely the same event

$$\mathcal{E}(\mathbf{z}, c): \qquad \mathsf{Comp}_d(\mathbf{u}) = \mathsf{Comp}_d(\mathbf{u} - \alpha c). \tag{$\star$}$$

In particular, when expressed in terms of $(\mathbf{z}, c)$, the acceptance event is independent of the hidden bit b. Consequently, conditioning on $\mathcal{E}$ does not introduce the branch-dependent bias that would arise if only one value of b were subject to a nontrivial compression-stability test.

The simulator samples

$$\mathbf{z} \leftarrow \mathcal{D}_{\mathbf{z}}, \qquad c \leftarrow U(\mathcal{C}),$$

computes

$$\mathbf{u} = \mathbf{A}\lfloor \mathbf{z} \rceil + \frac{q+1}{2}\alpha c \pmod q,$$

and restarts unless the event $\mathcal{E}(\mathbf{z}, c)$ in $(\star)$ holds. It then sets

$$\omega_1^{\mathsf{sim}} := \mathsf{Comp}_d(\mathbf{u}).$$

We next show that this value agrees with the first message in every accepting real execution. If $b = 0$, then

$$\mathbf{u} = \omega + \alpha c,$$

and the modified compression-stability condition gives

$$\mathsf{Comp}_d(\omega) = \mathsf{Comp}_d(\omega + \alpha c) = \mathsf{Comp}_d(\mathbf{u}).$$

If $b = 1$, then $\mathbf{u} = \omega$, and hence

$$\mathsf{Comp}_d(\omega) = \mathsf{Comp}_d(\mathbf{u}).$$

Thus, in both cases,

$$\omega_1^{\mathsf{real}} = \omega_1^{\mathsf{sim}} = \mathsf{Comp}_d(\mathbf{u}). \tag{1}$$

We now consider the hint $\mathbf{h}$. Using the decomposition

$$\mathbf{A}\lfloor \mathbf{z} \rceil = \mathbf{A}\lfloor \mathbf{z}_1 \rceil + \lfloor \mathbf{z}_2 \rceil,$$

we have

$$\mathbf{A}\lfloor \mathbf{z}_1 \rceil + \frac{q+1}{2}\alpha c = \mathbf{u} - \lfloor \mathbf{z}_2 \rceil \pmod q.$$

Therefore, the simulator computes

$$\mathbf{h}^{\mathsf{sim}} = \mathsf{Comp}_d(\mathbf{u}) - \mathsf{Comp}_d\big(\mathbf{u} - \lfloor \mathbf{z}_2 \rceil\big). \tag{2}$$

When $b = 0$, since $\mathbf{u} = \omega + \alpha c$, the hint in the real protocol satisfies

$$\begin{aligned}\mathbf{h}^{\mathsf{real}} &= \mathsf{Comp}_d(\omega + \alpha c) - \mathsf{Comp}_d\big(\omega + \alpha c - \lfloor \mathbf{z}_2 \rceil\big) \\ &= \mathsf{Comp}_d(\mathbf{u}) - \mathsf{Comp}_d\big(\mathbf{u} - \lfloor \mathbf{z}_2 \rceil\big).\end{aligned}$$

When $b = 1$, since $\mathbf{u} = \omega$, we similarly obtain

$$\begin{aligned}\mathbf{h}^{\mathsf{real}} &= \mathsf{Comp}_d(\omega) - \mathsf{Comp}_d\big(\omega - \lfloor \mathbf{z}_2 \rceil\big) \\ &= \mathsf{Comp}_d(\mathbf{u}) - \mathsf{Comp}_d\big(\mathbf{u} - \lfloor \mathbf{z}_2 \rceil\big).\end{aligned}$$

Hence, for every accepting execution,

$$\mathbf{h}^{\mathsf{real}} = \mathbf{h}^{\mathsf{sim}}. \tag{3}$$

The remaining components of the compressed response, including

$$\mathbf{x} = \mathsf{Encode}\big(\mathsf{Comp}_d(\lfloor \mathbf{z}_1 \rceil)\big)$$

and

$$\mathbf{e}_z = \lfloor \mathbf{z}_1 \rceil - \mathsf{Decomp}_d\big(\mathsf{Comp}_d(\lfloor \mathbf{z}_1 \rceil)\big),$$

are deterministic functions of $\mathbf{z}_1$. They therefore have the same conditional distribution in the real and simulated executions.

Let

$$\rho := \Pr_{\substack{\mathbf{z} \leftarrow \mathcal{D}_{\mathbf{z}} \\ c \leftarrow U(\mathcal{C})}} [\mathcal{E}(\mathbf{z}, c)].$$

Assuming that ρ is non-negligible, the expected number of iterations of the simulator is $1/\rho$, and hence the simulator runs in expected polynomial time. Since both the real protocol and the simulator condition the same underlying distribution on the same event $\mathcal{E}(\mathbf{z}, c)$, and all transcript components are subsequently computed by the same deterministic functions of $(\mathbf{z}, c)$, the data-processing inequality gives

$$\Delta\left(\mathsf{View}_{\mathsf{real}}^{\mathsf{acc}}, \mathsf{View}_{\mathsf{sim}}\right) \leq \frac{2\varepsilon_{\mathsf{RS}}}{\rho - \varepsilon_{\mathsf{RS}}},$$

provided that $\rho > \varepsilon_{\mathsf{RS}}$. Thus, whenever $\varepsilon_{\mathsf{RS}}$ is negligible and ρ is non-negligible, the accepting real and simulated transcripts are statistically indistinguishable.

In the special case where the rejection-sampling procedure produces $\mathcal{D}_{\mathbf{z}}$ exactly, we have $\varepsilon_{\mathsf{RS}} = 0$, and consequently

$$\mathsf{View}_{\mathsf{real}}^{\mathsf{acc}} \equiv \mathsf{View}_{\mathsf{sim}}.$$

That is, the accepting real and simulated transcripts are identically distributed. This completes the proof.

Commitment Guessing Probability. Let $m = nk$, and define the raw and compressed commitments as

$$\Omega := A\lfloor Y \rceil \bmod q, \qquad W := \mathrm{Comp}_d(\Omega),$$

where Y is sampled according to the commitment algorithm and Comp_d is applied coefficient-wise.

For a fixed public key pk, let

$$\delta_{pk} := \Delta\big(\mathcal{L}(\Omega \mid pk), U(\mathbb{Z}_q^m)\big)\,,$$

where $\Delta(\cdot,\cdot)$ denotes statistical distance and $U(\mathbb{Z}_q^m)$ denotes the uniform distribution over $\mathbb{Z}_q^m$.

Lemma 1 (Commitment guessing probability). For every fixed key pair (pk, sk), the commitment guessing probability satisfies

$$\epsilon(pk, sk) := \max_{w_0 \in \mathbb{Z}_{2^d}^m} \Pr[W = w_0 \mid pk, sk] \leq \left(\frac{\lceil q/2^d \rceil}{q}\right)^m + \delta_{pk}.$$

Equivalently, the min-entropy of the compressed commitment is bounded by

$$H_\infty(W \mid pk, sk) \geq -\log_2\left(\left(\frac{\lceil q/2^d \rceil}{q}\right)^m + \delta_{pk}\right).$$

Proof. Let U be uniformly distributed over $\mathbb{Z}_q^m$. For a single coefficient, every element in the range of

$$\mathrm{Comp}_d : \mathbb{Z}_q \longrightarrow \mathbb{Z}_{2^d}$$

has at most $\lceil q/2^d \rceil$ preimages. Since compression is applied coefficient-wise, every $w_0 \in \mathbb{Z}_{2^d}^m$ has at most $\lceil q/2^d \rceil^m$ preimages. Therefore,

$$\Pr[\mathrm{Comp}_d(U) = w_0] \leq \left(\frac{\lceil q/2^d \rceil}{q}\right)^m.$$

Moreover, statistical distance cannot increase under a deterministic mapping. Hence,

$$\Delta(\mathcal{L}(\mathrm{Comp}_d(\Omega) \mid pk), \mathcal{L}(\mathrm{Comp}_d(U))) \leq \delta_{pk}.$$

By the event characterization of statistical distance, for every $w_0 \in \mathbb{Z}_{2^d}^m$,

$$\begin{aligned}\Pr[W = w_0 \mid pk, sk] &\leq \Pr[\mathrm{Comp}_d(U) = w_0] + \delta_{pk} \\ &\leq \left(\frac{\lceil q/2^d \rceil}{q}\right)^m + \delta_{pk}.\end{aligned}$$

Taking the maximum over all w_0 proves the claimed bound on $\epsilon(pk, sk)$. The min-entropy bound follows from

$$H_\infty(W \mid pk, sk) = -\log_2 \max_{w_0} \Pr[W = w_0 \mid pk, sk].$$

□

In particular, suppose that δ_{pk} is negligible for all but a negligible fraction of public keys generated by KeyGen(1^λ). Then

$$\epsilon(pk, sk) \leq \left(\frac{\lceil q/2^d \rceil}{q}\right)^{nk} + \delta_{pk}$$

is negligible. If, in addition, the non-abort probability $1 - P(pk, sk)$ is inverse-polynomial in λ, then all the additional terms involving $\epsilon(pk, sk)$ and $P(pk, sk)$ in Eq. (12) are negligible.

3.3.2 UF-CMA Security

In this subsection, the unforgeability of our scheme is demonstrated in two steps. First, we prove the security under the UF-NMA model. We then extend the reduction to establish full UF-CMA security.

Theorem 4. For any adversary $\mathcal{A}$ attacking the UF-NMA security of the new signature scheme with at most Q_h hash queries, there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ such that:

$$\mathrm{Adv}^{\mathrm{UF-NMA}}(\mathcal{A}) \leq \mathrm{Adv}^{\mathrm{MNLWE}}(\mathcal{B}) + \mathrm{Adv}^{\mathrm{BimodalSelfTargetMSIS}}(\mathcal{C}). \tag{10}$$

Proof. We begin by randomizing the public key of the signature scheme. Let the key generation algorithm KeyGen(1^λ) output

$$(\mathbf{A}_0 \mid \mathbf{A}_1 \mid \mathbf{I}) \leftarrow \mathrm{KeyGen}(1^\lambda),$$

and denote $\bar{\mathbf{A}} = (\mathbf{A}_0 \mid \mathbf{A}_1)$. Then $\bar{\mathbf{A}} \in R_q^{k \times l}$ is a part of public matrix. If an adversary can distinguish between $\bar{\mathbf{A}}$ generated by KeyGen and a uniformly random distribution over $\mathcal{R}_q^{k \times l}$, we can directly construct a distinguisher $\mathcal{B}$ to solve the MNLWE problem. Otherwise, with probability $\mathrm{Adv}^{\mathrm{UF-NMA}}(\mathcal{A})$, adversary $\mathcal{A}$ returns a signature $(\mathbf{z}_1, \mathbf{h}, c)$ for some message μ satisfying the verification equation

$$c = \mathsf{H}_2\left(\mathrm{Comp}_d\left(\bar{\mathbf{A}}\mathbf{z}_1 + \frac{q+1}{2}c \cdot \alpha\right) + \mathbf{h}, \mu\right).$$

Note that compression does not appear in the BimodalSelfTargetRSIS problem. Therefore, we first decompress the term $\mathrm{Comp}_d\left(\bar{\mathbf{A}}\mathbf{z}_1 + \frac{q+1}{2}c \cdot \alpha\right) + h$ to obtain

$$\begin{aligned}
&\mathrm{Decomp}_d\left(\mathrm{Comp}_d\left(\bar{\mathbf{A}}\mathbf{z}_1 + \frac{q+1}{2}c \cdot \alpha\right) + \mathbf{h}\right) \\
&= e_1 + \mathrm{Decomp}_d(\mathbf{h}) + \mathrm{Decomp}_d\left(\mathrm{Comp}_d\left(\bar{\mathbf{A}}\mathbf{z}_1 + \frac{q+1}{2}c \cdot \alpha\right)\right) \\
&= e_1 + \mathrm{Decomp}_d(\mathbf{h}) + \left(\bar{\mathbf{A}}\mathbf{z}_1 + \frac{q+1}{2}c \cdot \alpha\right) + e_2,
\end{aligned}$$

where e_1 is the error between the decompression of the sum and the sum of decompressions, and it can be shown that $\|e_1\|_\infty = 1$. The term e_2 is the error introduced by compression and subsequent decompression and $\|e_2\|_\infty \leq \left\lfloor \frac{q}{2^{d+1}} \right\rceil$

We define a new function H', noting that the only difference between H and H' lies in the input format. The verification equation can then be rewritten as:

$$c = \mathsf{H}'\left(\bar{\mathbf{A}}\mathbf{z}_1 + e_2 + \frac{q+1}{2}c \cdot \alpha + \mathrm{Decomp}_d(\mathbf{h}) + e_1, \mu\right). \tag{11}$$

Let $E = e_1 + e_2$. Then Eq. (11) provides a polynomial vector $Y = [\mathbf{z}_1, \mathrm{Decomp}_d(\mathbf{h}) + E]^T$ such that

$$\mathsf{H}'\left([\bar{\mathbf{A}}|\mathbf{I}] \cdot \mathbf{Y} + \frac{q+1}{2}c \cdot \alpha, \mu\right) = c,$$

After decompressing the hint $\mathbf{h}$, we obtain:

$$\begin{aligned}
\mathrm{Decomp}_d(\mathbf{h}) &= \mathrm{Decomp}_d(\mathrm{Comp}_d(\omega) - \mathrm{Comp}_d(\omega - \lfloor \mathbf{z}_2 \rceil + (1-b)c\alpha)) \\
&= \mathrm{Decomp}_d(\mathrm{Comp}_d(\omega)) + e_3 - \mathrm{Decomp}_d(\mathrm{Comp}_d(\omega - \lfloor \mathbf{z}_2 \rceil + (1-b)c\alpha)) \\
&= \omega + e_4 - (\omega - \lfloor \mathbf{z}_2 \rceil + (1-b)c\alpha) + e_5 + e_3 \\
&= \lfloor \mathbf{z}_2 \rceil - (1-b)c\alpha + (e_3 + e_4 + e_5) \\
&= \tilde{\mathbf{z}}_2 + e_3 + e_5.
\end{aligned}$$

Similarly, e_3 is the error between the decompression of the sum and the sum of decompressions. The term e_4 and e_5 are the error introduced by compression and subsequent decompression. From this decompression result, we can derive an upper bound for Y as follows:

$$\begin{aligned}\|\mathbf{Y}\|_2 &= \|(\mathbf{z}_1, \mathrm{Decomp}_d(\mathbf{h}) + E)\|_2 \\ &= \|(\mathbf{z}_1, \tilde{\mathbf{z}}_2 + e_3 + e_5 + E)\|_2 \\ &\leq \|(\mathbf{z}_1, \tilde{\mathbf{z}}_2)\|_2 + 2\|E\|_2 \\ &\leq B'' + \sqrt{nk}(\frac{q}{2^d} + 3).\end{aligned}$$

□

Theorem 5. [3] Let ID be a canonical identification scheme with commitment guessing probability $\epsilon_{(pk,sk)}$ and abort probability $P_{(pk,sk)}$. Assume ID satisfies the saHVZK property as Definition 5 . For any adversary $\mathcal{A}$ attacking the UF-CMA security of our signature scheme, making Q_s signing queries and Q_h hash queries, there exists an adversary $\mathcal{B}$ attacking the UF-NMA security of our signature scheme such that:

$$\begin{aligned}\mathrm{Adv}^{\mathrm{UF-CMA}}(\mathcal{A}) \leq &\mathrm{Adv}^{\mathrm{UF-NMA}}(\mathcal{B}) + \frac{2Q_s\sqrt{\epsilon_{(pk,sk)}}}{1-P_{(pk,sk)}}\sqrt{Q_h + 1 + \frac{Q_s}{1-P_{(pk,sk)}}} \\ &+ 2(Q_h+1)\sqrt{\frac{Q_s\epsilon_{(pk,sk)}}{1-P_{(pk,sk)}}},\end{aligned} \tag{12}$$

where abort probability $P(pk, sk) := \Pr_{(w,st)\leftarrow \mathrm{Com}(sk)}[\mathrm{Resp}(w, c, st) = \perp]$ and the guessing probability: $\epsilon_{(pk,sk)} := \max_{w_0 \in W} \Pr_{(w,st)\leftarrow \mathrm{Com}(sk)}[w = w_0]$

Combining Eq. (10) and Eq. (12) demonstrates that the proposed signature scheme is UF-CMA secure.

3.3.3 The Addition of the Strong Unforgeabilty Property

While UF-CMA security prevents forgeries on novel messages, sUF-CMA security further ensures that an adversary cannot produce a new signature for a previously signed message. To establish the sUF-CMA security of our scheme, Theorem 7 extends the UF-CMA framework by incorporating the CUR property of the underlying identification protocol. Specifically, if an adversary succeeds in generating a second valid signature for an already-signed message, a rewinding technique can be employed to extract two distinct valid transcripts for the same commitment, thereby breaking the CUR property. To this end, we first bound the adversary's advantage against the CUR property. We then prove that the DARTS scheme achieves sUF-CMA security assuming the hardness of the MNLWE and MSIS problems.

Theorem 6. Let $\mathcal{A}$ be an adversary against the CUR property of our signature scheme, then there exist adversaries $\mathcal{B}$ and $\mathcal{C}$, each using $\mathcal{A}$ once and performing negligible additional work, such that

$$\mathrm{Adv}^{\mathrm{CUR}}(\mathcal{A}) \leq \mathrm{Adv}^{\mathrm{MNLWE}}(\mathcal{B}) + \mathrm{Adv}^{\mathrm{MSIS}}_{q,k,l,2B''+\sqrt{nk}(\frac{q}{2^{d-1}}+6)}(\mathcal{C}) \tag{13}$$

Proof. First, we replace the public matrix $\bar{\mathbf{A}}$ with a uniformly random matrix. The indistinguishability of this change is guaranteed by the MNLWE assumption, following the same reduction as in Theorem 4. An adversary $\mathcal{A}$ against the CUR property will return $\omega, c, \mathbf{z}_1, \mathbf{h}, \mathbf{z}_1', \mathbf{h}'$ such that $(\mathbf{z}_1, \mathbf{h}) \neq (\mathbf{z}_1', \mathbf{h}')$ and both $\omega, c, \mathbf{z}_1, \mathbf{h}$ and $\omega, c, \mathbf{z}_1', \mathbf{h}'$ are accepting transcript for the scheme. Thus,

$$\mathbf{h} + \mathsf{Comp}_d\left(\bar{\mathbf{A}}\mathbf{z}_1 + \frac{q+1}{2}c\alpha\right) = \mathbf{h}' + \mathsf{Comp}_d\left(\bar{\mathbf{A}}\mathbf{z}_1' + \frac{q+1}{2}c\alpha\right) = \mathsf{Comp}_d(\omega).$$

Therefore, following the same method used to derived Eq. (11), we can get the following results after decompression, modulo q:

$$\bar{\mathbf{A}}\mathbf{z}_1 + \frac{q+1}{2}c\cdot\alpha + e_2 + \mathrm{Decomp}_d(h) + e_1 = \bar{\mathbf{A}}\mathbf{z}_1' + \frac{q+1}{2}c\cdot\alpha + e_2' + \mathrm{Decomp}_d(h) + e_1',$$

where e_1, e_1' is the error between the decompression of the sum and the sum of decompressions, and it can be shown that $\|(e_1, e_1')\|_\infty = 1$. The term e_2, e_2' is the error introduced by compression and

subsequent decompression and $\|(e_2, e_2')\|_\infty \leq \left\lfloor \frac{q}{2^{d+1}} \right\rceil$. It follows that $[\bar{\mathbf{A}} \mid \mathbf{I}] \cdot \mathbf{X} \equiv 0 \pmod q$ where $\mathbf{X} = \begin{bmatrix} \mathbf{z}_1 - \mathbf{z}_1' \\ \mathrm{Decomp}_d(\mathbf{h}) - \mathrm{Decomp}_d(\mathbf{h})' + e_1 + e_2 - e_1' - e_2' \end{bmatrix}$ and $\|\mathbf{X}\|_2 \leq 2B'' + \sqrt{nk}(\frac{q}{2^{d-1}} + 6)$. If $\mathbf{z}_1 \neq \mathbf{z}_1'$, then $\mathbf{X} \neq 0$. Hence, $\mathbf{X}$ is a solution to the MSIS problem. If $\mathbf{z}_1 = \mathbf{z}_1'$ and suppose that $e_2 = e_2'$, $\mathrm{Decomp}_d(\mathbf{h}) - \mathrm{Decomp}_d(\mathbf{h})' + e_1 + e_2 - e_1' - e_2' = \mathrm{Decomp}_d(\mathbf{h}) - \mathrm{Decomp}_d(\mathbf{h})' + e_1 - e_1'$. While $\mathbf{h} \neq \mathbf{h}'$ and $\|e_1 - e_1'\|_\infty \leq 2$, $\mathbf{X} \neq 0$. Therefore, in both cases, $\mathbf{X}$ is a non-zero short vector to solve the MSIS problem.

□

Theorem 7. For any probabilistic polynomial-time adversary $\mathcal{A}$ against the sUF-CMA security of DARTS, there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ such that:

$$\mathrm{Adv}^{\mathrm{sUF-CMA}}(\mathcal{A}) \leq \mathrm{Adv}^{\mathrm{UF-CMA}}(\mathcal{B}) + \mathrm{Adv}^{\mathrm{CUR}}(\mathcal{C}) \tag{14}$$

where $\mathrm{Adv}^{\mathrm{CUR}}(\mathcal{C})$ denotes the advantage of an adversary in breaking the Computational Unique Response of the underlying identification protocol.

Hence, combining with Eq. (10), Eq. (12), Eq. (13) and Eq. (14), we can conclude that DARTS is sUF-CMA security.

3.4 Analysis with respect to known attacks

According to Theorem 2, solving BimodalSelfTargetMSIS is at least as hard as solving Search-MSIS with the same ring and modulus. To analyze the security of BimodalSelfTargetMSIS, we further evaluate the computational hardness of its underlying MSIS instantiation. Given a Search-MSIS instance $\mathbf{A} \in \mathbb{Z}_q^{k \times l}$ and a bound $\beta > 0$, we define the lattices

$$\Lambda = \{\, \mathbf{y} \in \mathbb{Z}^{k+l} : (\mathbf{A} \mid \mathbf{I}_k) \cdot \mathbf{y} = \mathbf{0} \pmod q \,\}.$$

A solution to the MSIS problem is any non-zero vector $\mathbf{y}$ in Λ such that $0 < \|\mathbf{y}\|_2 < \beta$.

For the security analysis of MLWE, the standard approach is to estimate the cost of lattice-reduction attacks based on the BKZ (Block Korkine–Zolotarev) algorithm [18, 7]. Two of the most important attack strategies are the primal attack and the dual attack. The primal attack embeds the MLWE instance into a lattice and reduces it to a bounded-distance decoding (BDD) problem, which can be solved through a uSVP formulation. In contrast, the dual attack constructs a distinguisher using a short vector in the dual lattice. The details of these reductions are presented in the following sections.

Primal Attack The Primal Attack transforms the MLWE instance into a unique Shortest Vector Problem (uSVP). Given an MLWE instance $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}) \in \mathbb{Z}_q^{k \times l} \times \mathbb{Z}_q^k$, we construct a $(k + l + 1)$-dimensional embedding lattice $\Lambda \subset \mathbb{Z}^{k+l+1}$ defined as:

$$\Lambda = \{(\mathbf{x}, \mathbf{y}, t) \in \mathbb{Z}^k \times \mathbb{Z}^l \times \mathbb{Z} \mid \mathbf{x} \equiv \mathbf{y}\mathbf{A}^T + t\mathbf{b}^T \pmod q\}.$$

The attacker aims to find the target short vector $\mathbf{v} = (-\mathbf{e}^\top, \mathbf{s}^\top, -1) \in \Lambda$. Since $\|\mathbf{v}\|$ is significantly smaller than the expected length of other non-parallel vectors in Λ, the problem is solved by applying the BKZ algorithm to find this unique shortest vector.

Dual Attack The Dual Attack aims to find a short vector in the dual lattice to construct a distinguisher. An attacker searches for a short vector $(\mathbf{u}, \mathbf{v}) \in \mathbb{Z}^k \times \mathbb{Z}^l$ such that $\mathbf{u}^T\mathbf{A} + \mathbf{v}^T \equiv \mathbf{0} \pmod q$. By computing the inner product $z = \mathbf{u}^T\mathbf{b} = \mathbf{u}^T\mathbf{A}\mathbf{s} + \mathbf{u}^T\mathbf{e} \equiv -\mathbf{v}^T\mathbf{s} + \mathbf{u}^T\mathbf{e} \pmod q$, the attacker can distinguish the MLWE samples from a uniform distribution. The security strength is determined by the shortest vector found via the BKZ algorithm on the dual lattice.

Dimension for Free Following the methodology in [10], we account for the "Dimension for Free" (D4F) gain to ensure a precise security assessment. The number of saved dimensions, denoted as k, is determined by the heuristic formula:

$$k = b \cdot \frac{\ln(4/3)}{\ln(b/2\pi e)}.$$

The core mechanism of the BKZ algorithm involves repeatedly calling an SVP Oracle on a smaller block size b to achieve global reduction of the lattice basis. Regarding the choice of the Oracle, we

compare enumeration and sieving. Enumeration requires very little memory. However, its time complexity grows super-exponentially with the dimension. Therefore, this method is only useful in low-dimensional settings. In contrast, sieving requires exponential memory. However, its time efficiency is much better in high dimensions. In terms of asymptotic performance, sieving is far superior to enumeration. With the advancement of memory performance and parallelization techniques, sieving has become the benchmark oracle for security assessments in both academia and the NIST standardization process.

To provide a simplified yet conservative estimate of security strength, the Core-SVP model is commonly adopted. This model ignores the number of BKZ iterations and polynomial factors, focusing solely on the computational cost of a single invocation of the most potent SVP oracle on block size b. Furthermore, we also consider the "Dimension for Free" [10] technique. It ensures that an attacker solving a b-dimensional lattice only incurs a cost equivalent to a $(b-k)$-dimensional problem, where k denotes the number of dimensions saved. Ultimately, based on best-known sieving complexities, the security strength of MLWE is quantified as $2^{0.292b}$ [4] in classical computing environments and $2^{0.257b}$ [6] in quantum environments via quantum-accelerated sieving.

Consequently, Table 3 provides a comprehensive summary of the security strengths for our proposed parameter sets, accounting for various attack vectors and the dimensions-for-free optimization.

.

Table 3: Security srength

Security Level	**128 bits**	**256 bits**	**512 bits**
Forgery: SIS Hardness (Core-SVP)			
BKZ block-size b (GSA)	470	895	1832
Classical Core-SVP	137	261	535
Quantum Core-SVP	120	230	470
Key-recovery: LWE Hardness (Core-SVP)			
Primal Attack			
BKZ block-size b (GSA)	560	934	1981
Classical Core-SVP	163	273	560
Quantum Core-SVP	143	240	492
Dual Attack			
BKZ block-size b (GSA)	550	922	1881
Classical Core-SVP	160	269	550
Quantum Core-SVP	141	236	483
Dimension for free [21]			
BKZ block-size b (GSA)	526	880	1797
Classical Core-SVP	153	256	524
Quantum Core-SVP	135	226	461

4 Performance Evaluation

This section discusses implementation aspects and reports performance results for the reference implementation, an AVX2-optimized implementation and a NEON-optimized one.

4.1 Reference implementation

To evaluate the performance of our signature scheme, we benchmark its reference implementation written in pure C and compiled using GCC 15.2.0 with `-O3 -march=native -mtune=native`. Table 4 and Table 5 present the performance results utilizing SM3 and FIPS 202, respectively. The evaluation is conducted on an `x86_64` platform with a 2.00 GHz AMD Ryzen AI 9 HX 370 processor. To guarantee deterministic results and minimize noise, we disable simultaneous multithreading (SMT) and fix the CPU frequency by turning off all auto-boosting features. The benchmark runs strictly on a single thread.

Table 4: **Reference implementation benchmarks with SM3.** Median and average cycle counts across 10,000 runs of DARTS. Results are obtained on a single core of an AMD Ryzen AI 9 HX 370 with SMT and core boosting disabled.

Scheme		KeyGen	Sign	Verify
DARTS-128	*med*	513800	7150760	203720
	ave	600391	9157683	217142
DARTS-256	*med*	641720	12221720	515640
	ave	761786	16009629	546057
DARTS-512	*med*	1271440	14682820	1013160
	ave	1309792	20746593	1067562

Table 5: **Reference implementation benchmarks with FIPS 202.** Median and average cycle counts across 10,000 runs of DARTS. Results are obtained on a single core of an AMD Ryzen AI 9 HX 370 with SMT and core boosting disabled.

Scheme		KeyGen	Sign	Verify
DARTS-128	*med*	276720	1250120	80120
	ave	322135	1655304	83800
DARTS-256	*med*	220140	2299140	161700
	ave	285119	3017369	174162
DARTS-512	*med*	494140	2611580	291880
	ave	522195	3890155	306744

Based on the experimental results in table 5, we provide the following analysis.

Key generation and rejection sampling efficiency Experimental results indicate that key generation incurs a non-trivial computational overhead, primarily due to the rejection sampling strategy applied to the secret key. However, from a systemic perspective, this cost remains negligible as key generation is typically a one-time operation. By conditioning the secret key through rejection sampling, DARTS effectively minimizes the rejection rate of the bimodal sampling process during signing. Consequently, this design successfully shifts the computational burden to the one-time setup phase, ensuring a high-efficiency signing routine with a low rejection rate.

Signing time analysis The signature phase is the most computationally intensive part of our scheme. Our benchmarks reveal that uniform random sampling over hyperballs emerges as the primary performance bottleneck, accounting for over 60% of the total signing time. These findings support the necessity of our optimization in key generation and bimodal rejection sampling over hyperballs.

Lightweight performance of verification The verification phase demonstrates high computational efficiency in our experiments due to a simplified process. In this process, the public key matrix is reconstructed via sample combinations derived from the given $\mathbf{A}_0$ and $\mathbf{A}_1$, where $\mathbf{A}_1$ is generated from seed$\mathbf{A}$. As a result, the experimental results show low CPU cycle counts. These findings confirm the scheme's suitability for resource-constrained environments. Consequently, DARTS satisfies the requirements of low-power hardware for rapid authentication.

4.2 AVX2 optimized implementation

The AVX2-optimized implementation is compiled using the same compiler with `-O3 -march=native -mtune=native -std=gnu99 -fcommon`. The system features 16 GB LPDDR5x RAM and runs Ubuntu 22.04 LTS on WSL2. The AVX2-optimized implementation leverages the AVX2 instruction set available on this processor. No external cryptographic library is used apart from the auxiliary hash function provided with the submission package.

The computational efficiency of our scheme is primarily dictated by the following core primitives: the Number Theoretic Transform(NTT), uniform hyperball sampling. To maximize the efficiency of our scheme on modern x86_64 architecture, we provide a highly optimized implementation leveraging Intel AVX2 intrinsics where we process eight 32-bit coefficients in parallel within 256-bit YMM registers. Table 6 and Table 7 present the performance gains of our AVX2-optimized implementation in terms of CPU cycles and energy efficiency. The experimental data shows that the execution time of core operations decreases significantly compared to the standard C implementation. This improvement is primarily due to the introduction of SIMD parallel instructions. These results further demonstrate the potential of our scheme for high-performance server environments.

Table 6: **AVX2 optimized implementation speeds of SM3.** Median and average cycle counts across 10,000 runs of DARTS. Results are obtained on a single core of an AMD Ryzen AI 9 HX 370 with SMT and core boosting disabled.

Scheme		**KeyGen**	**Sign**	**Verify**
DARTS-128	*med*	414620	8795020	167060
	ave	486003	11312588	178814
DARTS-256	*med*	515260	11421500	443980
	ave	629740	15557274	481185
DARTS-512	*med*	1004160	13516799	897500
	ave	1041052	19294779	940011

Table 7: **AVX2 optimized implementation speeds of FIPS 202.** Median and average cycle counts across 10,000 runs of DARTS. Results are obtained on a single core of an AMD Ryzen AI 9 HX 370 with SMT and core boosting disabled.

Scheme		**KeyGen**	**Sign**	**Verify**
DARTS-128	*med*	187880	701940	43460
	ave	224115	911434	46455
DARTS-256	*med*	107160	1099600	80440
	ave	146736	1449440	86181
DARTS-512	*med*	258180	1389600	151820
	ave	277600	1899994	165647

Integration of SM3 optimizations. The integration of AVX2-optimized SM3 primitives yields substantial improvements in key generation and verification of our signature scheme. Specifically, the Key Generation process is now 17.33%-20.52% faster, and the Verification speed is increased by 11.88%-17.65%. However, the 32-bit word-oriented design of SM3 involves serial dependencies between rounds, which limits the efficiency of SIMD vectorization. Without a four-way parallel interface, AVX2 is restricted to serial execution. This limits any performance gains in hyperball sampling and matrix $\mathbf{A}$ generation, with the former being the dominant bottleneck in the signing process. As a result, the signature scheme shows no significant speedup in this configuration.

Integration of FIPS 202 optimizations. Similarly, the FIPS 202 optimized version achieves significant efficiency gains. Compared to the reference implementation, the speedup distribution for the core components is as follows: the NTT is accelerated by 86.75%-87.52%, the uniform hyperball sampling by 15.87%-41.28%. These low-level optimizations directly translate into significant efficiency gains for the top-level algorithms. As indicated by the experiments, the Key Generation process is now 30.43%-48.54% faster, primarily due to the accelerated polynomial sampling and NTT-based matrix operations. The Signing procedure, which is the most computationally intensive phase, achieves a speedup of 44.94%-52.13% by leveraging parallelized rejection sampling within the hyperball. Finally, the Verification speed is increased by 44.56%-50.52%, benefiting from the vectorized polynomial arithmetic.

4.3 NEON optimized implementation

To evaluate the performance of our scheme on ARM-based platforms, we implement the core cryptographic primitives using ARM NEON vector instructions. Performance benchmarks are conducted on a single core of a Kunpeng 920 7280Z processor (2.90 GHz), equipped with 15 GiB of RAM. The deployment environment runs Ubuntu 22.04.5 LTS, and all code is compiled using GCC version 11.4.0. Table 8 reports the median and average clock cycle counts across 10,000 independent executions for each security level of DARTS.

Table 8: **NEON optimized implementation speeds of DARTS.** Median and average cycle counts of 10000 executions for DARTS. Cycle counts were obtained on one core of a Kunpeng 920 7280Z, running at 2900MHz with Ubuntu 22.04.5 LTS.

Scheme		**KeyGen**	**Sign**	**Verify**
DARTS-128	*med*	26013	394263	11071
	ave	28599	478988	11106
DARTS-256	*med*	32042	663072	27666
	ave	37587	840285	27772
DARTS-512	*med*	62574	683029	52558
	ave	62779	1071981	52728

5 Key Properties

Our scheme inherits the post-quantum security of lattice cryptography and achieves significant overall performance improvements. We will present some key properties as follows.

Novel Hardness Assumption We employ a new hardness assumption MNLWE, which can reduce from classical MLWE assumption. Hence, it provides a robust and mathematically grounded security foundation. By leveraging the specific structure of this assumption to generate a key pair, our scheme achieves higher security than other signatures based on bimodal rejection sampling under the same parameters. Based on this property, we appropriately sparsify the secret key distribution while maintaining a desired security level. As a result, the size of offset center is reduced, which lowers the number of rejections. Consequently, the signing speed is improved.

Flexible Parameter Choice A key advantage of the proposed scheme is its strong parameter flexibility. The scheme can adjust many parameters, such as the degree n of the polynomial ring, the modulus q, and the rejection bound B''. These parameters control the trade-off among security level, signature size, and computational cost. This tunable nature facilitates a linear transition across NIST Security Levels 1 to 5 within a unified algorithmic framework. It also enables high-level post-quantum security on high-performance cloud servers by simply adjusting the parameters. Such parametric agility optimizes real-world performance. Specifically, both the verification key size and the signature size of DARTS-128, which achieves 137-bit security, fit within the standard 1500-byte MTU, ensuring fragmentation-free, single-packet TCP transmission. Furthermore, the scalability ensures long-term adaptability. From a software engineering perspective, the core components—including rejection sampling, NTT, and hash mapping—are strictly decoupled. This modularity allows independent integration of more efficient cryptographic primitives as they emerge. The overall protocol logic is not affected.

Standard Compatibility Regarding compatibility, the design of DARTS fully considers the inheritance and adaptation of existing cryptographic standards and engineering practices. On one hand, the scheme complies with the parameter sets and API definitions prescribed in the ICCS call for proposals, ensuring rigorous compatibility across different protocol implementations. On the other hand, it employs a standard extendable-output function (XOF) as a core primitive. By leveraging the ability of an XOF to generate a pseudorandom stream of arbitrary length, the scheme efficiently performs matrix generation and coefficient sampling. Furthermore, the implementation does not rely on specific hardware for random number generation and can directly reuse existing general-purpose cryptographic computing

libraries. This design significantly lowers the barriers for system integration and the costs of protocol migration, demonstrating substantial practical utility.

References

[1] Bai, S., Beard, A., Johnson, F., KB Vidhanalage, S., Ngo, T.: Fiat-shamir signatures based on module-ntru. In: Australasian Conference on Information Security and Privacy. pp. 289–308. Springer (2022)

[2] Bai, S., Jangir, H., Lin, H., Ngo, T., Wen, W., Zheng, J.: Compact encryption based on module-ntru problems. In: International Conference on Post-Quantum Cryptography. pp. 371–405. Springer (2024)

[3] Barbosa, M., Barthe, G., Doczkal, C., Don, J., Fehr, S., Grégoire, B., Huang, Y.H., Hülsing, A., Lee, Y., Wu, X.: Fixing and mechanizing the security proof of fiat-shamir with aborts and dilithium. In: Annual International Cryptology Conference. pp. 358–389. Springer (2023)

[4] Becker, A., Ducas, L., Gama, N., Laarhoven, T.: New directions in nearest neighbor searching with applications to lattice sieving. In: Proceedings of the twenty-seventh annual ACM-SIAM symposium on Discrete algorithms. pp. 10–24. SIAM (2016)

[5] Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: Crystals-kyber: a cca-secure module-lattice-based kem. In: 2018 IEEE European symposium on security and privacy (EuroS&P). pp. 353–367. IEEE (2018)

[6] Chailloux, A., Loyer, J.: Lattice sieving via quantum random walks. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 63–91. Springer (2021)

[7] Chen, Y., Nguyen, P.Q.: Bkz 2.0: Better lattice security estimates. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 1–20. Springer (2011)

[8] Cheon, J.H., Choe, H., Devevey, J., Güneysu, T., Hong, D., Krausz, M., Land, G., Möller, M., Stehlé, D., Yi, M.: Haetae: Shorter lattice-based fiat-shamir signatures. IACR Transactions on Cryptographic Hardware and Embedded Systems 2024(3), 25–75 (2024)

[9] Devevey, J., Fawzi, O., Passelègue, A., Stehlé, D.: On rejection sampling in lyubashevsky's signature scheme. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 34–64. Springer (2022)

[10] Ducas, L.: Shortest vector from lattice sieving: a few dimensions for free. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 125–145. Springer (2018)

[11] Ducas, L., Durmus, A., Lepoint, T., Lyubashevsky, V.: Lattice signatures and bimodal gaussians. In: Annual Cryptology Conference. pp. 40–56. Springer (2013)

[12] Duda, J.: Asymmetric numeral systems: entropy coding combining speed of huffman coding with compression rate of arithmetic coding. arXiv preprint arXiv:1311.2540 (2013)

[13] Espitau, T., Tibouchi, M., Wallet, A., Yu, Y.: Shorter hash-and-sign lattice-based signatures. In: Annual International Cryptology Conference. pp. 245–275. Springer (2022)

[14] Gärtner, J.: Ntwe: A natural combination of ntru and lwe. In: International Conference on Post-Quantum Cryptography. pp. 321–353. Springer (2023)

[15] Langlois, A., Stehlé, D.: Worst-case to average-case reductions for module lattices. Designs, Codes and Cryptography 75(3), 565–599 (2015)

[16] Lyubashevsky, V.: Fiat-shamir with aborts: Applications to lattice and factoring-based signatures. In: International conference on the theory and application of cryptology and information security. pp. 598–616. Springer (2009)

[17] Lyubashevsky, V.: Lattice signatures without trapdoors. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 738–755. Springer (2012)

[18] Schnorr, C.P., Euchner, M.: Lattice basis reduction: Improved practical algorithms and solving subset sum problems. Mathematical programming 66(1), 181–199 (1994)

[19] Voelker, A.R., Gosmann, J., Stewart, T.C.: Efficiently sampling vectors and coordinates from the n-sphere and n-ball. Centre for Theoretical Neuroscience-Technical Report 1(2017) (2017)

[20] Woo, J., Kim, J., Hong, G.H., Lee, S., Kim, M., Lee, H., Park, J.H.: Ntru+ sign: compact ntru-based signatures using bimodal distributions. Cryptology ePrint Archive (2025)

[21] Xia, W., Wang, L., Wang, G., Gu, D., Wang, B.: A refined hardness estimation of lwe in two-step mode. In: IACR International Conference on Public-Key Cryptography. pp. 3–35. Springer (2024)

[22] Zhu, Y., Liu, Z., Pan, Y.: When ntt meets karatsuba: preprocess-then-ntt technique revisited. In: International conference on information and communications security. pp. 249–264. Springer (2021)